Mustrid

Versioon 1.52 (30. august 2025)

Sisukord

Sissejuhatus ... 2
Terminoloogiast ... 2
Lugemise juhend ... 3
Nimi: Põhiolemite hetkeseisundi registreerimise vajadus ... 4
Allsüsteemid ... 5
Kontseptuaalne andmemudel ... 6
Seisundidiagramm ... 8
Andmebaasioperatsioonide lepingud ... 12
Kasutusjuhud ... 20
Tegevusdiagramm ... 22
CRUD maatriks ... 24
Nimi: Põhiolemite hetkeseisundi registreerimine SQL-andmebaasis ... 26
Nimi: Põhiolemite seisundimuudatuste võimaldamine graafilises kasutajaliideses ... 31
Kasutuaja arvutis olev rakendus (MS Accessi näitel) ... 32
Veebirakendus (Oracle APEX näitel) ... 37
Nimi: Põhiolemite teatavate seisundimuudatuste keelamine SQL-andmebaasis ... 47
MS Access (andmemakrod) ... 47
Oracle Enterprise Edition (trigerid) ... 50
Nimi: Põhiolemite seisundimuudatuste ajaloo registreerimine ... 53
Edasiseks lugemiseks ... 60

Sissejuhatus

Palun lugege seda enne, kui asute mustritega tutvuma.

Terminoloogiast

Olem on reaalses maailmas eksisteeriv füüsiline või abstraktne asi (objekt, eksemplar) – näiteks inimene nimega Erki Eessaar või Tallinna Tehnikaülikooli õppeaine nimega „Andmebaasid I“.

Olemitüüp on reaalses maailmas esinevate olemite üldistus. Näiteks inimesed Erki Eessaar, Juhan Juurikas, Mari Maasikas ja teised üldistuvad olemitüübiks *Isik* ning õppeained „Andmebaasid I“, „Andmebaasid II“ ja teised üldistuvad olemitüübiks *Aine*. Teiste sõnadega on olemitüüp nime omav lõplik hulk sarnaste omaduste ja käitumisega olemeid.

Infosüsteemi **põhiolemitüübid** (mõnikord kasutatakse terminit **põhiobjektid**) kirjeldavad põhilisi andmeid millega süsteemi vahendusel on võimalik töötada. Süsteemides, kus põhirõhk on andmete/info töötlemisel, on need nagu sambad, millele kogu süsteem toetub. Kõik tegevused, mida süsteemis saab teha on seotud nende andmete töötlemisega (lisamise, lugemise, muutmise ja kustutamisega).

Infosüsteemi põhiolemitüüp on selline olemitüüp, mis täidab kõiki järgnevaid nõudeid.

- Sellesse kuulub rohkem kui üks olem.
- Sellesse kuuluvad olemid on seotud infosüsteemi valdkonnaga (pakuvad infosüsteemi mõttes huvi).
- Sellesse kuuluvate olemite kohta käivaid andmeid on vaja infosüsteemis säilitada ja sellest tulenevalt hallata (lisada, lugeda, muuta, kustutada).
- Sellesse kuuluvatel olemitel on infosüsteemi jaoks huvi pakkuv elutsükkel (elukaar) ning sellest tulenevalt hargnevad infosüsteemis protsessid nende olemite ümber.
- Sellest võivad sõltuda teiste olemitüüpide (ei ole põhiolemitüübid) olemid.
- Seda võib kasutada infosüsteemi jagamiseks alamosadeks e allsüsteemideks, mis on pühendatud vastavat tüüpi olemite kohta käivate andmete haldamisele.

Käesoleva materjali mustrite näidetes käsitletakse kahte põhiolemitüüpi – *Kaup* ja *Klassifikaator*.

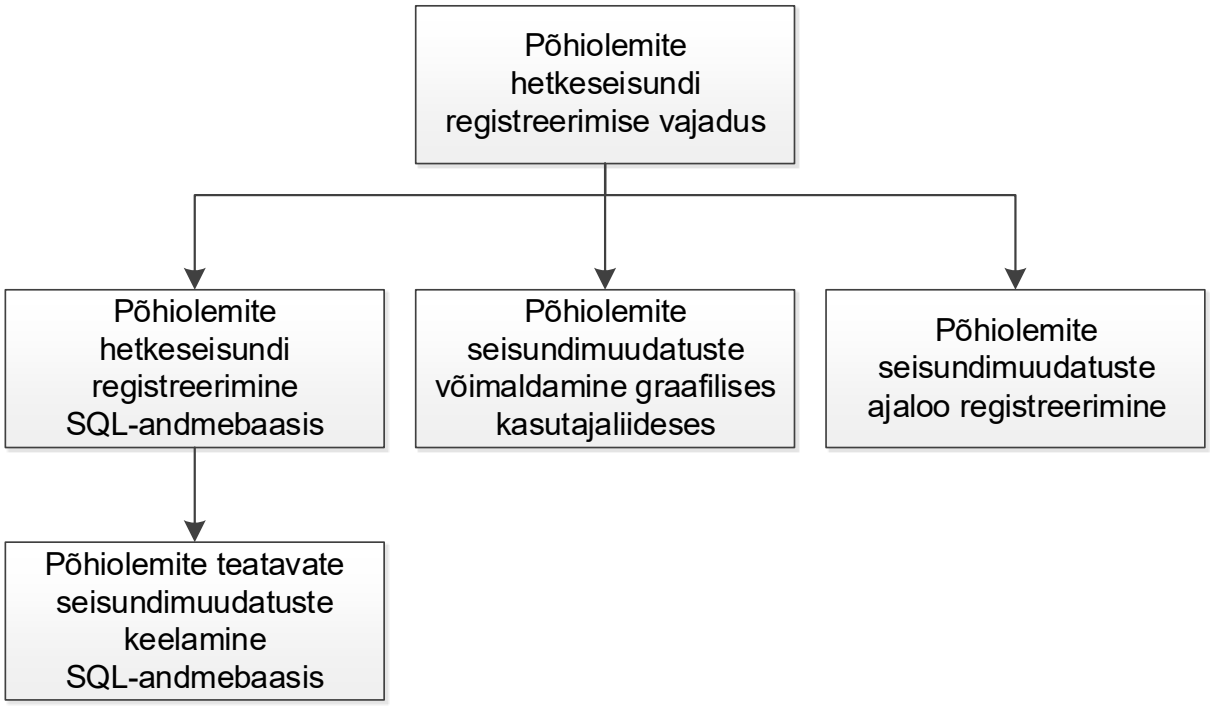
Muster on struktureeritud viisil kirja pandud kirjeldus probleemist ja selle heast ning levinud lahendusest. Mustrite nimedest moodustub sõnavara, mida kasutades saab probleemidele ja nende lahendustele viidata.

Lugemise juhend

Käesolevas dokumendis esitatakse mustrid, mis kirjeldavad, kuidas tulla toime vajadusega registreerida infosüsteemis põhiolemite seisund.

Järgnevalt esitatakse materjalis väljatoodud mustrite nimed ja nende omavahelised sõltuvused. Nool mustrist A mustrisse B (A→B) näitab, et mustri A rakendamine on mustri B rakendamise eelduseks.

Kui seda materjali otsast lõpuni läbi lugeda, siis määravad nooled ka soovitava lugemise järjekorra. Kõigepealt tuleks tutvuda mustriga *Põhiolemite hetkeseisundi registreerimise vajadus* ning seejärel vabalt valitud järjekorras järgnevate mustritega – *Põhiolemite hetkeseisundi registreerimine SQL-andmebaasis*, *Põhiolemite seisundimuudatuste võimaldamine graafilises kasutajaliideses*, *Põhiolemite seisundimuudatuste ajaloo registreerimine*. Enne mustriga *Põhiolemite teatavate seisundimuudatuste keelamine SQL-andmebaasis* tutvumist tuleks lugeda mustrit *Põhiolemite hetkeseisundi registreerimine SQL-andmebaasis*.



Nimi: Põhiolemite hetkeseisundi registreerimise vajadus

Ingliskeelne nimi: The need to register the current state of the main entities

Eelmised mustrid: -

Probleem: Kuidas modelleerida infosüsteemi detailanalüüsi dokumendis vajadust säilitada andmebaasis andmeid põhiolemitüüpi kuuluvate olemite (põhiolemite) hetkeseisundi kohta?

Kontekst:

- Andmebaas peab olema projekteeritud nii, et see võimaldaks vastata korrektselt formuleeritud päringutele antud probleemvaldkonnas nii praegu kui ka tulevikus.
- Tüüpiliste päringutega soovitakse teada, milline on mingi põhiolemi hetkeseisund või millised põhiolemid on mingis kindlas seisundis.
- Põhiolemiga tehtavad võimalikud tegevused ja selle reaktsioon keskkonnas toimuvatele sündmustele sõltuvad põhiolemi hetkeseisundist, kuid mitte seisunditest, milles see olem oli eelnevalt.
- Iga põhiolemitüübi korral peab andmebaas võimaldama registreerida vastavat tüüpi olemite hetkeseisundi.
- Põhiolemitüüpi kuuluvate olemite võimalike seisundite hulk võib äriprotsesside muutumisest tulenevalt aja jooksul muutuda.
- Põhiolemitüüpi kuuluvate olemite seisundi ajaloo säilitamine ei ole oluline.

Lahendus: Modelleerida iga põhiolemitüübi kohta kontseptuaalses andmemudelis seisundi klassifikaator. Kuna igale põhiolemitüübile vastab infosüsteemi äirarhitektuuris eraldi register, siis tuleks üldjuhul näha ette üks seisundi klassifikaator iga registri kohta. Klassifikaatorite (sealhulgas seisundi klassifikaatorite) andmed on eraldi registris – *klassifikaatorite registris*. **NB!** *Klassifikaator* on üks põhiolemitüüp ning ka klassifikaatorite puhul on oluline registreerida nende väärtuste hetkeseisund, kuid jätame selle praegu lihtsuse huvides vaatluse alt välja.

Lisaks tuleb iga põhiolemitüübi korral modelleerida seisundidiagrammi e olekumasinaskeemi abil sellesse tüüpi kuuluvate põhiolemite kõikvõimalikud *infosüsteemi jaoks huvi pakkuvad* seisundid ja seisundi üleminekud. Võib juhtuda, et „päris maailmas“ on põhiolemitel mõni selline elutsükli etapp (seisund), mis infosüsteemis huvi ei paku. Näiteks piletimüügi infosüsteemi kavandamisel võidakse otsustada, et ei paku huvi, kas üritus toimus või mitte (kaks eraldi lõppseisundit), vaid ainult see, et ürituse toimumisaeg on möödunud ja sellele enam pileteid müüa ei saa (üks lõppseisund). Sellisel juhul erineks „päris maailma“ kohta koostatav elutsükli mudel (selline koostatakse ärianalüüsi käigus) ja infosüsteemi jaoks

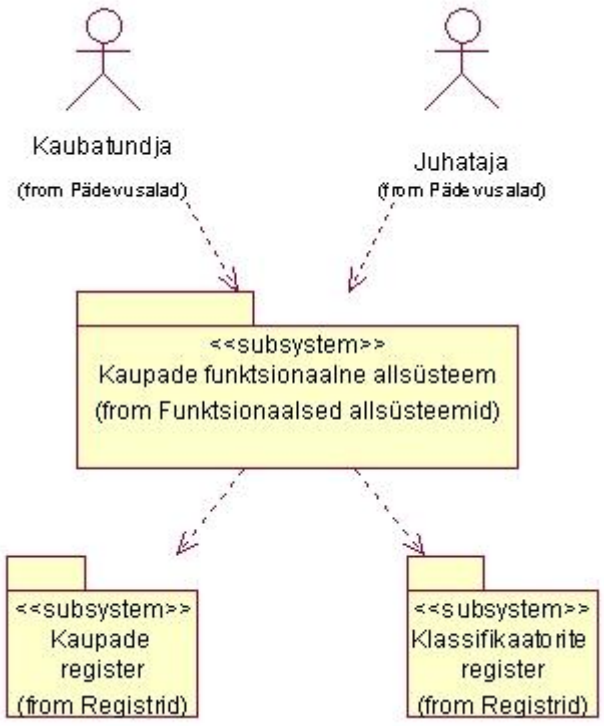
koostatav elutsükli mudel (selline koostatakse süsteemianalüüsi käigus ning on esitatud käesoleva mustri näites). Üldiselt peaks infosüsteem peegeldama võimalikult hästi kõike seda olulist ja vajalikku, mis eksisteerib pärismaailmas. Seega tasub taolisi lihtsustusi teha võimalikult vähe, kui üldse. Näiteks ürituste näite puhul ei säili süsteemis info ürituste ebaõnnestumiste kohta ja seega pole ka selge, milliste ürituste piletid tuleks tagasi osta. Kui mingi korraldaja üritused lõpevad sageli ebaõnnestumisega, siis võiks piletimüügifirma parema teenuse ja parema maine huvides loobuda selle korraldaja üritustele piletite müümisest või küsida korraldajalt lisagarantiisid. Kui süsteemis ei säili info, millised üritused ebaõnnestusid, siis pole ka täpset „mälu“, mille põhjal sellist läbimõeldud otsust teha.

Seisundidiagramm esitab põhiolemite kõikvõimalikud elutsüklid. Erinevatel põhiolemitel on erinev elutsükkel (nagu näiteks inimestel on erinev elusaatus), kuid diagramm võtab kõik need kokku. Seisundidiagrammil esitatud seisundite alusel leitakse seisundi klassifikaatori väärtused. Üldjuhul vastab igale seisundile (välja arvatud algne pseudoseisund) eraldi klassifikaatori väärtus. Kui üleminekul seisundisse „Kustutatud“ või „Unustatud“ kustutatakse põhiolemi andmed andmebaasist, siis pole seisundile „Kustutatud“ või „Unustatud“ vastavat eraldi klassifikaatori väärtust vaja. Kuna andmebaasis tuleb registreerida põhiolemite hetkeseisund, siis vastab igale seisundidiagrammil esitatud seisundi üleminekule andmebaasioperatsioon, mis tuleb kirjeldada lepingu formaati kasutades. Iga kirja pandud operatsiooni korral peab leiduma vähemalt üks kasutusjuht e kasutusmall, mis seda operatsiooni kasutab (välja kutsub). Seega on seisundidiagramm ja andmebaasioperatsioonide lepingud aluseks kasutusjuhtude mudeli koostamisele. Kasutusjuhtude abil väljendatud protsesse saab modelleerida tegevusdiagrammide abil.

Näide: Vaatleme näitena kaupluse infosüsteemi, mille korral on üheks põhiolemitüübiks *Kaup.* Kauba all mõeldakse antud juhul kaubaartiklit (kauba omaduste kirjeldust), mitte kauba eksemplari (füüsilist eset).

Allsüsteemid

Põhiolemitüübile *Kaup* vastab infosüsteemi äriarhitektuuris *kaupade register*. Äriarhitektuuri kohaselt teenindab *kaupade registrit* (lisab/loeb/muudab/kustutab selles andmeid) *kaupade funktsionaalne allsüsteem*. *Kaupade funktsionaalne allsüsteem* peab toimimiseks lugema andmeid ka teistest registritest – antud näite korral *klassifikaatorite registrist*. *Kaupade funktsionaalne allsüsteem* pakub teenuseid *kaubatundja* ja *juhataja* pädevusalade esindajatele. Joonis 1 esitatakse infosüsteemi äriarhitektuuri fragment, kasutades UML paketidiagrammi e paketiskeemi. Diagrammile on paigutatud tegutsejatena esitatud pädevusalad ning pakettidena esitatud funktsionaalsed allsüsteemid ja registrid. Nooled näitavad sõltuvusi.



Joonis 1 Kaupluse infosüsteemi äriarhitektuuri fragment.

Kontseptuaalne andmemudel

Joonis 2 esitab lihtsustatud *kaupade registri* olemi-suhte diagrammi e olemi-suhte skeemi, millel esitatakse põhiolemitüüp *Kaup* ja sellega seotud seisundi klassifikaator *Kauba_seisundi_liik*. *Kauba_seisundi_liik* pärib kõik *Klassifikaator* atribuudid. Taolises andmebaasis tuleb iga registreeritud kauba korral muuhulgas registreerida selle kauba hetkeseisund. Kui kauba seisund muutub, siis tuleb uus seisund andmebaasis registreerida ja see kirjutab vana seisundi üle (st seisundite ajalugu antud näite korral ei säili). Kui soovite säilitada seisundite ajalugu, siis vaadake palun mustrit *Põhiolemite seisundimuudatuste ajaloo registreerimine*.

NB! Lisaks olemi-suhte diagrammidele tuleb süsteemianalüüsi käigus koostada ka olemitüüpide ja atribuutide definitsioonid.

Näiteks, mida peab saama teada kauba hetke hinna atribuudi definitsioonist?

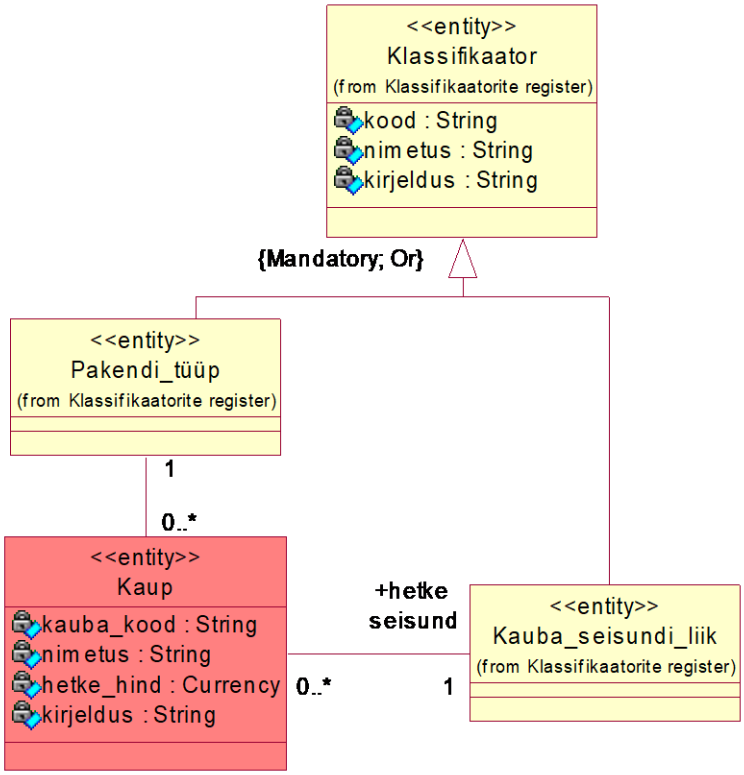
1. Kas hind on sisseostuhind või väljamüügihind?
2. Kas hind sisaldab käibemaksu? (Te ju ei taha, et andmeid haldav programm ning selle kasutajad jätavad käibemaksu arvestamata või arvestavad seda kaks korda!)
3. Mis valuutas on hind?

4. Kas hind on valuuta põhiühikus (nt eurodes) või hoopis selle alamühikus (nt sentides)? Märkus: tahtmine hindu alamühikutes salvestada võib tekkida sellest, et süsteemi realiseerimiseks kasutatavas keskkonnas toimuvad operatsioonid täisarvudega kiiremini kui püsikomaarvudega. See on näide, kuidas mõnikord tuleb teha disaini käigus valikuid, mida on vaja ka analüüsi mudelis väljendada.
5. Kui hind on põhiühikus, siis mitu kohta peale koma on hinna väärtustes maksimaalselt lubatud?
6. Kas hind võib olla negatiivne? Aga 0? Kas hinna väärtusel on ülapiir? Kui jah, siis milline? Milline on kõige suurem hind, mille registreerimise vajadust osatakse süsteemis ette näha?
7. Kas hinna registreerimine on kohustuslik?

Punktid 1-4 esitavad hinna väärtust esitava arvu juurde käivat ning kasutajale mõeldud interpretatsiooni. Süsteem ei oska seda kontrollida. Süsteemile pole vahet, kas registreeritav arv on sisseostuhind või kinganumber. Küll aga on see oluline lõppkasutajale ja tema peab kaupade haldamise vormi väärtust sisestades või seal sellist väärtust nähes teadma, mida see väärtus tähendab. Kasutajaliides peab andma selleks piisavat infot (vt mustrit *Põhiolemite seisundimuudatuste võimaldamine graafilises kasutajaliideses*). Samuti on see info oluline arendajatele, kes koostavad andmebaasi põhjal päringuid või realiseerivad andmebaasi kasutamiseks mõeldud andmebaasirakendust.

Punktid 5-7 on kitsendused, mille täidetust saab süsteem kontrollida. Atribuutide definitsioonis tuleb need panna looksulgudesse **{ }** ning süsteem realiseerida nii, et see ei laseks registreerida nende kitsendustega mitte kooskõlas olevaid andmeid. Punktides 4-7 olev info on muuhulgas vajalik andmebaasis atribuudile vastava kõige sobivama andmetüübi valimiseks. Hea viis selliste kitsenduste jõustamiseks oleks andmebaasis loodud kitsendused, sest nii pole võimalik nendest kitsendustest mööda hiilida.

Äri viib läbi protsesse, protsessidest tekivad jäljed andmete näol, mis registreeritakse andmebaasis. Kui andmete registreerimine lükatakse tarkvara poolt tagasi, sest andmed eksisid mõne kitsenduse vastu, siis jõuab ka äriprotsessi läbiviijateni teadmine, et nad tegid midagi valesti. Selle kaudu mõjutab tarkvara ja andmebaasid lõppkokkuvõttes äriprotsesside läbiviimist.

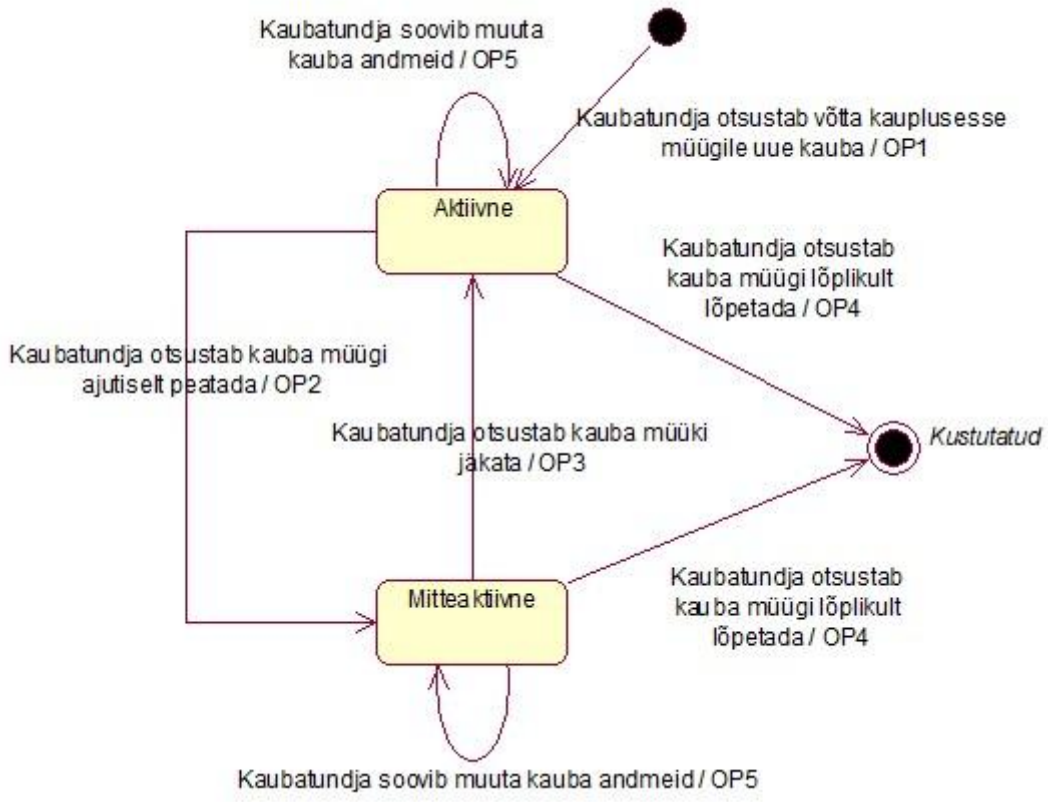


Joonis 2 Lihtsustatud kaupade registri olemi-suhte diagramm.

NB! Pange ka palun tähele, et olemitüübil *Kaup* puudub atribuut *kogus* või *laoseis*. Äriarhitektuuris on ka *laoliikumiste funktsionaalne allsüsteem* ja *laoliikumiste register*. Iga laoliikumine on seotud konkreetse kaubaga, mille andmed on *kaupade registris* ja mille andmeid hallatakse *kaupade funktsionaalses allsüsteemis*. Kaup on selles mõttes nagu *kauba konto* ning laoliikumised on nagu sissetulekud ja väljaminekud sellelt kontolt. Kontoseisu (antud juhul laoseisu) saab teada, liites kokku kõik sissetulekud ning lahutades kõik väljaminekud. Selle analoogiks on kontoseis pangas, mida võite näiteks internetipanka sisse logides vaadata.

Seisundidiagramm

Põhiolemite *elutsüklid* (kõikvõimalikud seisundid ja seisundi üleminekud) infosüsteemi kontekstis tuleb modelleerida seisundidiagrammi kasutades. Joonis 3 esitatakse põhiolemitüübi *Kaup* seisundidiagramm. Võib öelda, et seisundidiagramm väljendab põhiolemitüübi andmetega töötamise *töövoogu*.



Joonis 3 Kauba seisundidiagramm.

Kuna kauba seisundisse „Kustutatud“ viimisel ei kustutata kauba andmeid andmebaasist, siis on seisundi klassifikaatoril *Kauba_seisundi_liik* kolm väärtust (vt Tabel 1). Nagu näete, pärinevad klassifikaatori väärtuste nimetused ja kirjeldused olemitüübi *Kaup* seisundidiagrammist.

Tabel 1 Kauba seisundi klassifikaatori väärtused

seisundi_liik_kood	nimetus	kirjeldus
1	Aktiivne	Kaup jõuab sellesse seisundisse, kui kauplusesse võetakse müügile uus kaup või kui otsustatakse jätkata ajutiselt peatatud kauba müüki.
2	Mitteaktiivne	Kaup jõuab sellesse seisundisse, kui kauba müük otsustatakse ajutiselt peatada.
3	Kustutatud	Kaup jõuab sellesse seisundisse kui kauba müük otsustatakse lõplikult lõpetada. Kuna tegemist on lõppseisundiga, siis selles seisundis kaup ei saa süsteemis sama identiteediga taassündida. Teiste sõnadega, kui tuleb välja, et kaupa saab ikkagi

seisundi_liik_kood	nimetus	kirjeldus
		edasi müüa, siis tuleb süsteemis registreerida uus kaup.

Kui kasutaja annab kauba kustutamise korralduse, siis võib talle kasutajaliidese vahendusel jätta mulje, et andmed on kustutatud, kuid tegelikkuses muutus vaid kauba seisund. Võib öelda, et tegemist on pehme e loogilise kustutamisega, mis peidab küll kauba andmed laiema avalikkuse pilkude eest, kuid süsteem ei unusta kaupa ja selle andmed säilivad. Selle vastandiks on andmete füüsiline e reaalne kustutamine andmebaasist.

Miks vältida *andmete* füüsilist kustutamist?

- Andmete säilitamist võivad nõuda seadused või määrused. Näide:
  - „(1) Raamatupidamise algdokumente peab raamatupidamiskohustuslane säilitama seitse aastat alates selle majandusaasta lõpust, kui majandustehing algdokumendi alusel raamatupidamisregistris kirjendati.
  - (2) Raamatupidamisregistreid, lepinguid, raamatupidamise aruandeid ja muid äridokumente, mis on vajalikud majandustehingute arusaadavaks kirjeldamiseks revideerimise käigus, peab raamatupidamiskohustuslane säilitama seitse aastat, alates vastava majandusaasta lõpust.
  - (3) Pikaajaliste kohustiste või õigustega seotud äridokumente tuleb säilitada seitse aastat pärast kehtimistähtaja möödumist.
  - (4) Raamatupidamise sise-eeskirja tuleb säilitada seitse aastat pärast selle muutmist või asendamist.
  - (5) Käesoleva paragrahvi lõigetes 1−4 nimetatud dokumente ja andmeid säilitatakse masintöödeldavalt. Säilitatavate dokumentide ja andmete taasesitatavus kirjalikus vormis, lihttekstis loetavus ja tõendusväärtus peavad olema tagatud kogu säilitustähtaja jooksul. Dokumente ja andmeid võib säilitada muus püsivat kirjalikku taasesitamist võimaldavas vormis, kui raamatupidamiskohustuslasel ei ole dokumentide ja andmete masintöödeldavalt säilitamise võimalust ning selle võimaluse loomine nõuab temalt ebaproportsionaalselt suuri kulutusi või pingutusi.“ (Raamatupidamise seadus §12, https://www.riigiteataja.ee/akt/RPS)
- Andmed on vara. Mida pikema perioodi kohta andmeid koguda, seda rohkem on võimalik nendest andmetest erinevate andmeanalüüsi meetodite abil uut informatsiooni „välja pigistada“. Kui koguda andmeid piisavalt pika perioodi kohta, siis on võimalik erinevate andmekaeve meetodite abil hakata otsima andmetest peidetud seoseid ja trende ning isegi prognoosida tulevikku. Saadud informatsioon on oluline andmebaasi kasutava organisatsiooni edasist arengut puudutavate otsuste langetamisel ning võib anda eelise võrreldes konkureerivate organisatsioonidega. Andmekaevandajad ütlevad selle kohta, et andmed on uus nafta/energiaallikas.

- *Kaup* kuulub *põhiandmete* hulka. Põhiandmed (ingl *master data*) on andmed, mis kirjeldavad abstraktseid või füüsilisi „asju“, mis määravad transaktsiooniliste (tehingute) andmete konteksti. Kuigi antud näide seda ei kajasta, on kogu infosüsteemi arvestades kaupadega seotud väga palju transaktsioonilisi andmeid – näiteks tellimused, müügid, garantiijuhtumid, laoliikumised, inventuurid, kommenteerimised ja hindamised. Konkreetse kauba andmebaasist kustutamisel oleks vaja kustutada kõik sellega seotud transaktsioonilised andmed ning eelmist kahte punkti arvestades oleks see lubamatu.

Samas võivad seadused nõuda ka andmete kustutamist. Näide isikuandmete kohta:

- „§ 17. Isikuandmete säilitamine
  - (4) Kui isikuandmete säilitamise tähtaeg lõpeb, on vastutav ja volitatud töötleja kohustatud isikuandmed jäädavalt kustutama. Selleks peab vastutav töötleja rakendama nõuetekohaseid õiguslikke ja tehnoloogilisi meetmeid.
- § 25. Andmesubjekti õigus nõuda isikuandmete parandamist ja kustutamist
  - (3) Andmesubjektil on õigus vastutavalt töötlejalt nõuda kogutud isikuandmete kustutamist, kui:
    - 1) isikuandmete töötlemine ei ole seaduse alusel lubatud;
    - 2) isikuandmete töötlemisel ei arvestatud isikuandmete töötlemise põhimõtteid või
    - 3) vastutav töötleja on kohustatud andmed kustutama täitmaks seadusest, kohtuotsusest, välislepingust või muust siduvast kokkuleppest tulenevat kohustust.“ (Isikuandmete kaitse seadus, https://www.riigiteataja.ee/akt/IKS)

Veel üks näide – karistuste kohta:

„§ 24. Registrisse kantud karistusandmete kustutamise tähtajad

(1) Karistusandmed kustutatakse registrist ja kantakse üle arhiivi, kui:

1) väärteo eest mõistetud või määratud rahatrahvi tasumisest, aresti kandmisest, üldkasuliku töö sooritamisest või põhikaristusena juhtimisõiguse äravõtmisest on möödunud üks aasta;

2) maksukorralduse seaduses või maksuseaduses sätestatud väärteo eest mõistetud või määratud rahatrahvi tasumisest või aresti täitmisest on möödunud kaks aastat;

3) psühhiaatrilise sundravi lõpetamisest on möödunud kaks aastat;

4) sõltuvusravi kohaldamise lõpetamisest on möödunud kaks aastat;

5) kuriteo eest mõistetud rahalise karistuse otsuse täitmisest on möödunud kolm aastat;

6) tähtajalisest vangistusest või rahalisest karistusest tingimisi vabastamise korral määratud katseaja lõppemisest on möödunud kolm aastat;

7) üldkasuliku töö sooritamisest on möödunud kolm aastat;

8) alla viieaastase vangistuse ärakandmisest on möödunud viis aastat;

9) viie- kuni kahekümneaastase vangistuse ärakandmisest on möödunud kümme aastat;

10) üle kahekümneaastase vangistuse ärakandmisest on möödunud viisteist aastat;

11) otsuse täitmine on aegunud vastavalt karistusseadustiku §-le 82;

12) isik on surnud;

13) juriidiline isik on lõppenud.“ (Karistusregistri seadus, https://www.riigiteataja.ee/akt/KarRS)

Seaduseid tuleb täita – nende mittetundmine pole vabandus. Seega on õigusaktid oluline infosüsteemi nõuete allikas ja juristidel on kindlasti oluline roll infosüsteemide arendamise juures. Olukorda muudab keerulisemaks, kui tegemist on globaalse hajutatud süsteemiga, sest siis rakenduvad süsteemi erinevatele osadele erinevate riikide õigusaktid.

Erinevatel olemitel võivad olla erinevad elutsüklid. Näiteks kaubal k1 on see *Aktiivne=>Kustutatud*, kuid kaubal k2 on see *Aktiivne=>Mitteaktiivne=>Aktiivne=> Kustutatud*. Olemi järjestikulistest seisunditest moodustuv elutsükkel moodustab antud juhul Markovi ahela, millel on Markovi omadus. Markovi omadus tähendab, et olemi uus seisund sõltub tema hetkeseisundist, aga mitte sellest, millistes seisundites on see olem varem viibinud.

Andmebaasioperatsioonide lepingud

Kuna andmebaasis tuleb iga registreeritud kauba korral registreerida ka selle hetkeseisund, siis vastab igale kauba seisundidiagrammil esitatud seisundi üleminekule andmebaasioperatsioon, mis tuleb kirjeldada lepingu formaati kasutades. Iga sellise operatsiooni täitmise tulemusena registreeritakse konkreetse kauba uus seisund. Joonisel 3 on OP1–OP5 andmebaasioperatsioonide lepingute lühikesed identifikaatorid. Kuna kauba andmete kustutamisel tehakse sõltumata kauba eelnevast seisundist samasugused tegevused, siis panen kustutamise kohta kirja ühe operatsiooni (OP4).

Joonis 6 esitatud kasutusjuhtude diagrammis on kasutusjuht *Muuda kaupa* ning selle kasutusjuhu toetamiseks peab leiduma andmebaasioperatsioon, mille ülesandeks on muudetud kauba andmed andmebaasis registreerida. Selle operatsiooni tulemusena ei muutu kauba seisund – seisundi muutmiseks on eraldi operatsioonid. Seega juhime tähelepanu, et lisaks seisundit muutvatele operatsioonidele võib leiduda veel kauba andmete haldamisega seotud operatsioone (antud näite korral *OP5 Muuda kaupa*). Soovi korral võib seisundidiagrammis esitada üleminekud, kus algseisund ja lõppseisund on samad (rekursiivsed üleminekud). Nende abil saab esitada olulisi sündmuseid, millele olem vastavas seisundis reageerib, kuid mis seisundit ei muuda. Antud juhul väljendatakse nende abil, et „Aktiivne“ ja „Mitteaktiivne“ seisundis kaupu saab muuta. Kui sellised üleminekud muudavad seisundidiagrammi liiga kirjuks või kui neid soovitakse esitada lõppseisundite korral, kus need pole diagrammil lubatud, siis võib selguse huvides nende esitamisest diagrammil loobuda.

Järgnevalt esitatakse andmebaasioperatsioonid lepingu formaati kasutades. Operatsiooni *eeltingimused* näitavad operatsiooni väljakutsuja vastutusi – millised andmed peavad olema andmebaasis, et operatsioon saaks õnnestuda. Operatsiooni *järeltingimused* näitavad operatsiooni vastutusi – juhul kui eeltingimused on täidetud, siis millised andmemuudatused on andmebaasis operatsiooni tulemusena toimunud. Seda, mis jäi operatsiooni järel andmebaasis samaks, ei panda järeltingimustesse kirja (kui seda siiski teete tekitab see palju segadust ja vääritimõistmisi, sest lugeja arvab, et need andmed muutusid operatsiooni tulemusena). Seda, mis jääb andmebaasist väljapoole (näiteks kaup eemaldati füüsiliselt riiulitelt või andmebaasirakenduse kasutajaliideses kadus võimalus kauba andmeid muuta), ei panda samuti andmebaasioperatsioonide lepingusse kirja.

Operatsiooni edukaks täitmiseks peavad olema täidetud selle *kõik* eeltingimused. Operatsiooni eduka täitmise tulemuseks on *kõigi* selle järeltingimuste saavutamine.

Kontseptuaalne andmemudel on nagu raam (karkass, raamistik, tellingud) mille põhjal andmebaasioperatsioonide lepinguid kirja pannakse. Andmebaasioperatsioonide lepingute eel- ja järeltingimustes viidatakse operatsiooni parameetritele ja kontseptuaalses andmemudelis olevatele olemitüüpidele, atribuutidele ja seosetüüpidele. Teiste sõnadega, andmebaasioperatsioonide lepingutes tuleb kasutada sama terminoloogiat (olemitüüpide ja atribuutide nimed), mida kasutatakse kontseptuaalses andmemudelis, sest muidu pole võimalik aru saada nende mudelite omavahelistest seostest.

Sisendparameetrid (lühidalt *parameetrid*) määravad, millised andmed tuleb väljakutsutavale operatsioonile ette anda. Iga parameeter on muutuja – erinevatel väljakutsetel võib selle väärtus (argument) olla erinev (st muutuda). Operatsiooni väljakutsel on vaja anda parameetritele väärtused, sest operatsioon ise ei saa neid andmeid leida, kuna need jäävad andmebaasist väljapoole (näiteks kasutajaliidesesse) ja andmebaasis ei ole neid tagastavat operaatorit. Samas on andmeid nagu hetke kuupäev või hetke kuupäev + kellaaeg, mida saab küsida andmebaasis sees (pöördudes sobiva funktsiooni poole) ja seda pole vaja parameetri väärtusena operatsioonile ette anda.

Pange tähele, et nende operatsioonide parameetrite hulgas ei ole *p_kauba_seisundi_liigi_kood*, sest iga operatsiooni tulemusena läheb konkreetne kaup ühte kindlasse seisundisse, mis on kirjeldatud selle operatsiooni järeltingimustes. Seega ei ole operatsiooni väljakutsel vaja kauba seisundi liigi koodi operatsioonile ette anda.

Lähtuge parameetrite leidmisel minimaalsuse põhimõttest – nii palju kui vaja, nii vähe kui võimalik. Kui parameetrile lepingu eel- või järeltingimustes kordagi ei viidata, siis on tegemist veaga. Võimalik, et mõni eel- või järeltingimus on puudu. Võimalik, et parameeter on üleliigne.

OP1 Registreeri kaup (

p_kauba_kood,
p_nimetus,
p_hetke_hind,
p_kirjeldus,
p_pakendi_tüübi_kood

)

Eeltingimused:

- Kauba_seisundi_liik eksemplar ksl (millel on kood=1 ("Aktiivne")) on registreeritud
- Pakendi_tüüp eksemplar pt (millel on kood=**p_pakendi_tüübi_kood**) on registreeritud

Järeltingimused:

--Loo eksemplare

- Kaup eksemplar k on registreeritud

--Väärtusta atribuute

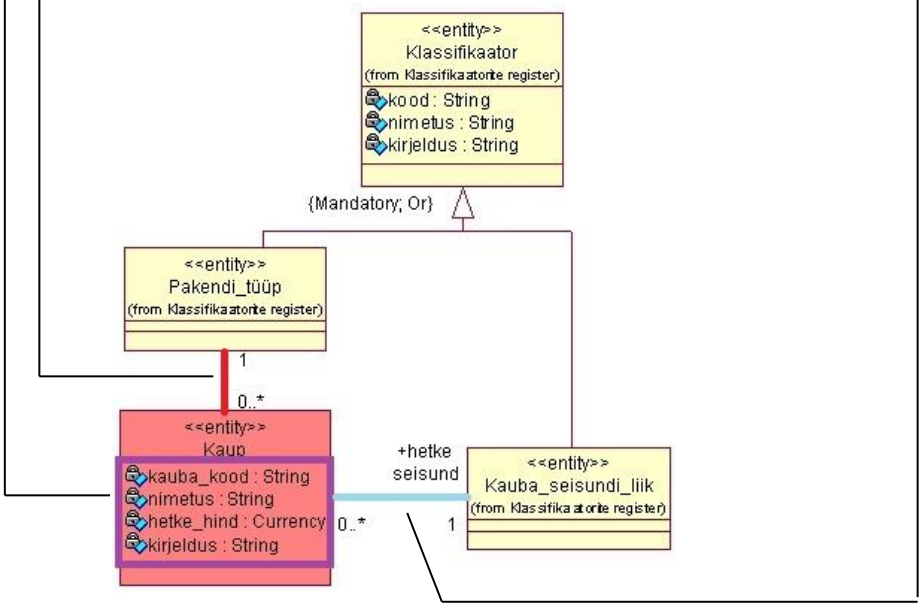
- k.kauba_kood:= **p_kauba_kood**
- k.nimetus:= **p_nimetus**
- k.hetke_hind:= **p_hetke_hind**
- k.kirjeldus:= **p_kirjeldus**

--Loo seoseid

- k ja ksl seos on registreeritud
- k ja pt seos on registreeritud

Kasutus kasutusjuhtude poolt: Registreeri kaup

Parameetrite ja järeltingimuste seoste mõistmiseks pöörake palun tähelepanu värvidele!ele!



Joonis 4 Olemi-suhte diagrammi ja andmebaasioperatsiooni lepingu OP1 kooskõla (vaadake värve ja nooli).

Programmeerimise taustaga inimesed võivad mõelda operatsioonide lepingutest kui protseduuri või teenuse *pseudokoodist*. Eeltingimuses deklareeritakse tähistus (OP2 korral eksemplaride identifikaatorid k, ksl_vana ja ksl_uus), mida kasutatakse järeltingimustes. Programmeerimisega tuttavad võivad sellest mõelda kui muutujate deklareerimisest – programmi alguses deklareeritakse muutujad, millele hiljem deklareeritud nime kasutades viidatakse. Kui nimi on vale, siis tekib viga. Sama on ka siin, ainult et programmivea asemel ei saa operatsiooni lugeja operatsioonist aru. See teeb sisuliselt sama välja, sest programm/leping ei täida endale seatud ülesannet. Igale parameetrile viidatakse eel- või järeltingimustes. Jällegi tuleb jälgida, et eel- ja järeltingimuses kasutatud parameetri nimi on samasugune nagu operatsiooni nime järel olevas parameetrite nimekirjas.

```
OP2 Muuda kaup mitteaktiivseks (
     p_kauba_kood
)
Eeltingimused:
• Kaup eksemplar k (millel on kauba_kood= p_kauba_kood) on
  registreeritud
• k on seotud Kauba_seisundi_liik eksemplariga ksl_vana
  (millel on kood=1 ("Aktiivne"))
• Kauba_seisundi_liik eksemplar ksl_uus (millel on kood=2
  ("Mitteaktiivne")) on registreeritud
Järeltingimused:
--Kustuta seoseid
• k ja ksl_vana seos on kustutatud
--Loo seoseid
• k ja ksl_uus seos on registreeritud
Kasutus kasutusjuhtude poolt: Muuda kaup mitteaktiivseks
```

OP3 Muuda kaup aktiivseks (
p_kauba_kood
)

Eeltingimused:

- Kaup eksemplar k (millel on kauba_kood= **p_kauba_kood**) on registreeritud
- k on seotud Kauba_seisundi_liik eksemplariga ksl_vana (millel on kood=2 ("Mitteaktiivne"))
- Kauba_seisundi_liik eksemplar ksl_uus (millel on kood=1 ("Aktiivne")) on registreeritud

Järeltingimused:

--Kustuta seoseid

- k ja ksl_vana seos on kustutatud

--Loo seoseid

- k ja ksl_uus seos on registreeritud

Kasutus kasutusjuhtude poolt: Muuda kaup aktiivseks

OP4 Kustuta kaup (
p_kauba_kood
)

Eeltingimused:

- Kaup eksemplar k (millel on kauba_kood= **p_kauba_kood**) on registreeritud
- k on seotud Kauba_seisundi_liik eksemplariga ksl_vana ((millel on kood=1 ("Aktiivne")) või (millel on kood=2 ("Mitteaktiivne")))
- Kauba_seisundi_liik eksemplar ksl_uus (millel on kood=3 ("Kustutatud")) on registreeritud

Järeltingimused:

--Kustuta seoseid

- k ja ksl_vana seos on kustutatud

--Loo seoseid

- k ja ksl_uus seos on registreeritud

Kasutus kasutusjuhtude poolt: Kustuta kaup

```
OP5 Muuda kaupa (
     p_kauba_kood_vana,
     p_kauba_kood_uus,
     p_nimetus,
     p_hetke_hind,
     p_kirjeldus,
     p_pakendi_tüübi_kood
)
Eeltingimused:
• Kaup eksemplar k (millel on kauba_kood=p_kauba_kood_vana) on
  registreeritud
• k on seotud Kauba_seisundi_liik eksemplariga ksl ((millel on
  kood=1 ("Aktiivne")) või (millel on kood=2
  ("Mitteaktiivne")))
• Pakendi_tüüp eksemplar pt (millel on
  kood=p_pakendi_tüübi_kood) on registreeritud
Järeltingimused:
--Väärtusta atribuute
• k.kauba_kood:= p_kauba_kood_uus
• k.nimetus:= p_nimetus
• k.hetke_hind:= p_hetke_hind
• k.kirjeldus:= p_kirjeldus
--Kustuta seoseid
• k olemasolev seos pakendi tüübiga on kustutatud
--Loo seoseid
• k ja pt seos on registreeritud
Kasutus kasutusjuhtude poolt: Muuda kaupa
```

Interaktiivne leht nende lepingutega tutvumiseks ja seoste paremaks nägemiseks: https://maurus.ttu.ee/interaktiivsus/kaupade_haldus/

NB! Soovitan eelnevatele lepingutele lisaks lugeda kohe ka mustrit *Põhiolemite seisundimuudatuste võimaldamine graafilises kasutajaliideses*, et saada aru, kuidas analüüsi käigus kirjapandavatest andmebaasioperatsioonide lepingutest tekivad andmebaasis rutiinid (protseduurid või funktsioonid) ning andmete muutmise laused ning kuidas lepingud tuleb panna kirja nii täpselt, et neist oleks võimalik koodi tuletada. **NB!** Seda koodi võib ka genereerida generatiivne tehisintellekt, millele antakse ette leping ning andmebaasi andmestruktuuride kirjeldus.

Miks on OP5 parameetrite hulgas kauba kood kaks korda? *Kaup* atribuut *kauba_kood* omab inimkasutajatele tähendust – nad kasutavad seda kaupadele viitamiseks ka väljaspool infosüsteemi tarkvara. Seetõttu on see ka modelleeritud kontseptuaalses andmemudelis. Kaupade koodid on olulised, sest nende alusel saavad näiteks tarnijad aru, millist kaupa soovib kauplus neilt tellida. Tarnija ja kauplus peavad kasutama ühesuguseid koode. Seega,

kauba kood registreeritakse inimkasutaja poolt, mitte ei genereerita süsteemi poolt. Inimene on ekslik või tahab ümberotsustada ja seega peab tal olema võimalus juba tehtut parandada. Seega peab saama muuhulgas muuta kaupade koode. Parameetri **p_kauba_kood_vana** väärtuse alusel leitakse üles muudetav kaup. Parameetri **p_kauba_kood_uus** väärtus saab leitud kauba uueks koodiks.

OP5 kirjeldatakse, kuidas saab muuta kaubaga seotud pakendi tüüpi. Igal kaubal peab olema täpselt üks pakendi tüüp. Analoogina mõelge elektrivõrgu-pistikupesadele. Iga kaubal on pakendi tüübi jaoks ainult üks pistikupesa (, sest vastava seosetüübi võimsustikud on: [Pakendi_tüüp]-**1**----0..*-[Kaup]). Üks konkreetne pakendi tüüp on nagu akulaadija, teine nagu sülearvuti. Akulaadija ja sülearvuti ei saa olla korraga samasse pistikupesasse ühendatud. Kui pistikupesas on parajasti akulaadija, siis pean selle kõigepealt pistikupesast välja võtma (kustuta seoseid) ning alles siis saan sülearvuti pistikupesasse ühendada (loo seoseid) (vt Joonis 5).

Joonis 5 Seoste loomine ja kustutamine.

Parameeter **p_pakendi_tüübi_kood** tähendab, et operatsioonile antakse väljakutsel ette pakendi tüübi kood. See identifitseerib unikaalselt ühte konkreetset pakendi tüüpi – seda pakendi tüüpi, millega soovitakse kaup muudatuse tulemusel siduda. Saame eeldada, et pakendi tüübil on kood, nimetus ja kirjeldus, sest *Pakendi_tüüp* on üldistusseose, mis ühtlasi on pärimisseos, abil modelleeritud *Klassifikaator* alamtüübina. *Kood*, *nimetus* ja *kirjeldus* on *Klassifikaator* atribuudid. Operatsioonile tuleb anda ette midagi, mis on iga pakendi tüübi korral unikaalne. Antud juhul kasutatakse pakendi tüübi koodi, mitte nimetust, sest see on lühem ja muutub harvem kui nimetus.

Kui selle asemel kasutada operatsiooni lepingus parameetrit **pakendi tüübi identifikaator**, siis see tähendaks, et operatsioonile antakse väljakutsel ette väärtus või väärtuste

kombinatsioon, mis identifitseerib unikaalselt ühte konkreetset pakendi tüüpi. Kuna pakendi tüübil võib olla mitu unikaalset identifikaatorit (antud juhul kood ja nimetus) ning tegemist on alles vajadusi paika paneva analüüsiga, siis tähendab see, et ei minda detailidesse ja ei määrata täpselt kindlaks, millist identifikaatorit kasutada. Seda tehakse alles disaini ja ehitamise käigus, kui selle andmebaasioperatsiooni tulemusena luuakse andmebaasiserveris talletatud rutiin. Sellisel kujul kirja pandud parameetri kasutamise põhjus võib olla ka selles, et analüüsi tulemusena ei leitud ühtegi olemitüübi sisulise tähendusega identifikaatorit ning rutiinides tuleb hakata kasutama kunstliku identifikaatorit (surrogaatvõtme väärtust), mis andmebaasis olemi andmetega seotakse.

Seega, kui on täpselt teada, et operatsioonile antakse ette pakendi tüübi kood, siis kasutan varianti 1, kui see pole kindel, siis lükkan otsustamise disaini ja kasutan varianti 2.

Iga operatsiooni õnnestunud täitmine (järeltingimuste saavutamine) loob eelduseid (täidab eeltingimusi) mingite teiste andmebaasioperatsioonide edukaks täitmiseks. Näiteks OP1 järeltingimuste saavutamine konkreetse kauba korral, täidab selle kauba haldamiseks OP2–OP5 püstitatud eeltingimuse.

- Kaup eksemplar k (millel on kauba_kood= **p_kauba_kood**) on registreeritud

Miks tuleb panna kirja eraldi operatsioon iga seisundi ülemineku kohta, selle asemel, et kirjeldada operatsioon *Kauba seisundi muutmine*, mille parameetriteks on *p_kauba_kood* ja *p_kauba_seisundi_liigi_kood*?

Kui igal seisundi ülemineku jaoks on omaette operatsioon, siis saab süsteemi realiseerides õiguste abil määrata, kellel on õigus milliseid seisundimuudatusi läbi viia. See omakorda aitab lihtsamalt saavutada süsteemi, kus kehtib kohustuste lahusus (https://en.wikipedia.org/wiki/Separation_of_duties), mis on kasulik vigade ja pettuste võimaluse vähendamiseks. Samuti on iga seisundi ülemineku kohta eraldi operatsiooni loomine paremini kooskõlas otstarbe lahususe põhimõttega (https://en.wikipedia.org/wiki/Separation_of_concerns), mis lihtsustab süsteemi muutmist. Näiteks oletame, et kerkib esile nõue, et mingi seisundimuudatuse korral on vaja registreerida täiendavaid andmeid, mida teiste seisundimuudatuste korral pole vaja teha. Saan muuta sellele seisundi üleminekule vastavat operatsiooni, ilma, et peaksin teisi seisundi üleminekutele vastavaid operatsioone muutma. Ühise operatsiooni korral muutuks selles olev loogika keerulisemaks (tingimuslaused) ning raskemini arusaadavaks ja hallatavaks.

Lepingprojekteerimise (https://en.wikipedia.org/wiki/Design_by_contract) käigus pannakse iga lepingu kohta kirja ka *invariandid* – need on tingimused, mis peavad olema täidetud nii enne operatsiooni algust kui ka peale operatsiooni lõppu. Analüüsi taseme andmebaasioperatsioonide lepingute korral on invariantideks *kitsendused* andmetele, mida

kirjeldatakse kontseptuaalses andmemudelis. Seega neid lepingu kirjelduses eraldi välja tooma ei hakata.

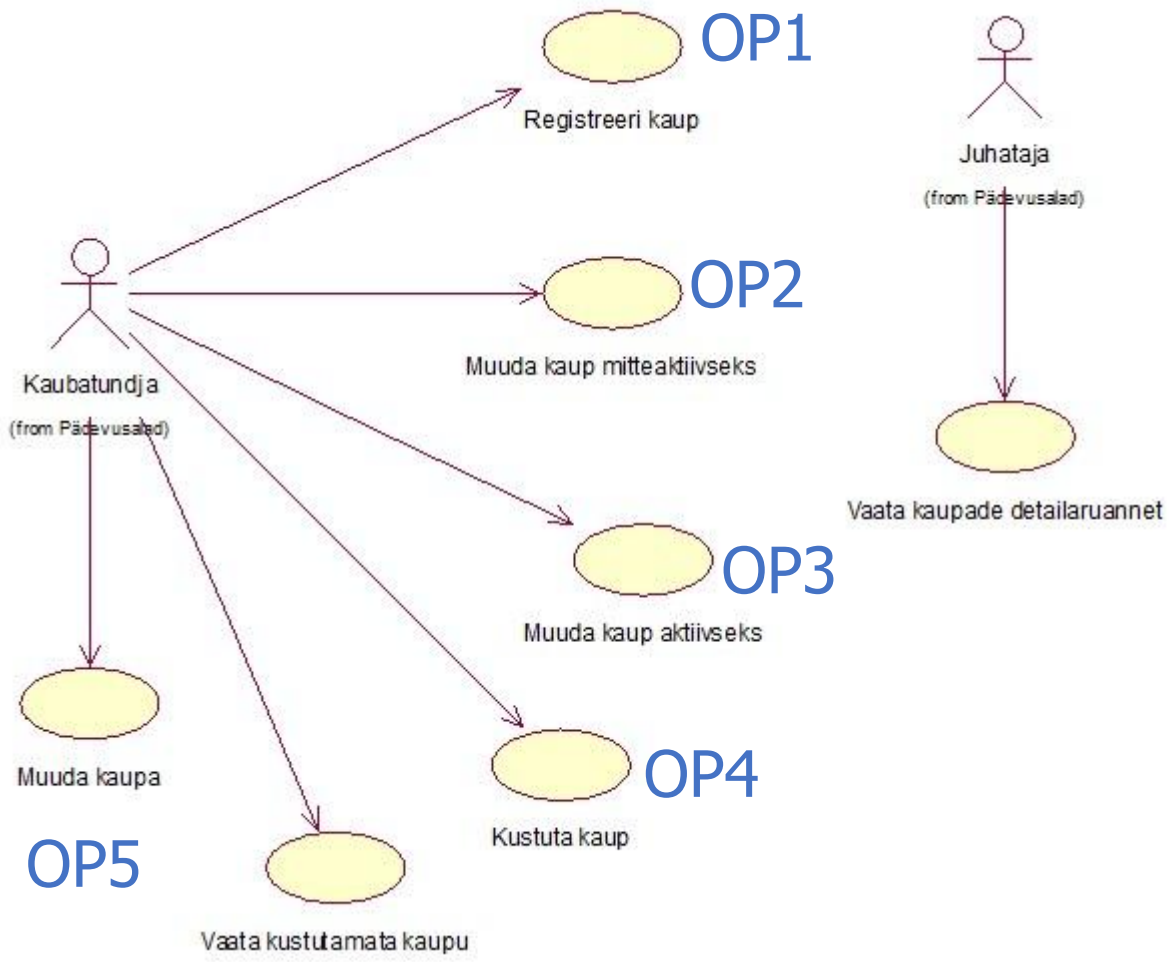
Kasutusjuhud

Operatsiooni lepingu üheks alamosaks on *kasutus kasutusjuhtude poolt*. Iga kirja pandud operatsiooni korral peab leiduma vähemalt üks kasutusjuht, mis seda operatsiooni kasutab (välja kutsub). Seega on seisundidiagramm ja andmebaasioperatsioonide lepingud aluseks kasutusjuhtude mudeli koostamisele. Käesoleva infosüsteemi äriarhitektuuri kohaselt teenindab *kaupade registrit kaupade funktsionaalne allsüsteem*. Joonis 6 esitatakse kaupade funktsionaalse allsüsteemi kasutusjuhtude diagramm e kasutusmalliskeem. Iga kasutusjuht on diagrammil seotud pädevusalaga (tegutsejaga), mille esindaja süsteemiga suhtleb, et seda kasutusjuhtu läbi viia.

Kasutusjuhtude juurde on kirjutatud viited andmebaasioperatsioonidele, mida selle kasutusjuhu täitmise käigus välja kutsutakse. Praktikas ei kirjutata viiteid andmebaasioperatsioonide lepingutele diagrammile, vaid lisatakse kasutusjuhtude tekstikirjeldustesse. Kasutusjuhtude mudel on eeskätt tekstiline mudel. Kasutusjuhtude tekstikirjeldused on nagu raamatu sisu ja kasutusjuhtude diagramm nagu raamatu sisukord. Lühiduse huvides käesolev materjal kasutusjuhtude tekstikirjeldusi ei sisalda.

Üldiselt vastab igale seisundidiagrammis esitatud seisundi üleminekule eraldi kasutusjuht. Siin on erandeid nagu antud näite korral kasutusjuht *Kustuta kaup*, mis võtab kokku kaks seisundi üleminekut samasse seisundisse. Nendel kasutusjuhtudele lisanduvad ainult andmete lugemiseks mõeldud kasutusjuhud nagu antud näites *Vaata kaupade detailaruannet* ja *Vaata kustutamata kaupu*.

Iga mudelis esitatud kasutusjuht vastab mingile elementaarsele äriprotsessile. Iga selline protsess viiakse läbi ühe tegutseja poolt ühel ajahetkel ja ühes kohas. Iga selline kasutusjuht käivitub reaktsioonina mingile sündmusele, selle läbimine annab tegutsejale väärtuse ning selle läbimise järel on süsteemis terviklikud andmed.



Joonis 6 Kaupade funktsionaalse allsüsteemi kasutusjuhtude mudeli fragment.

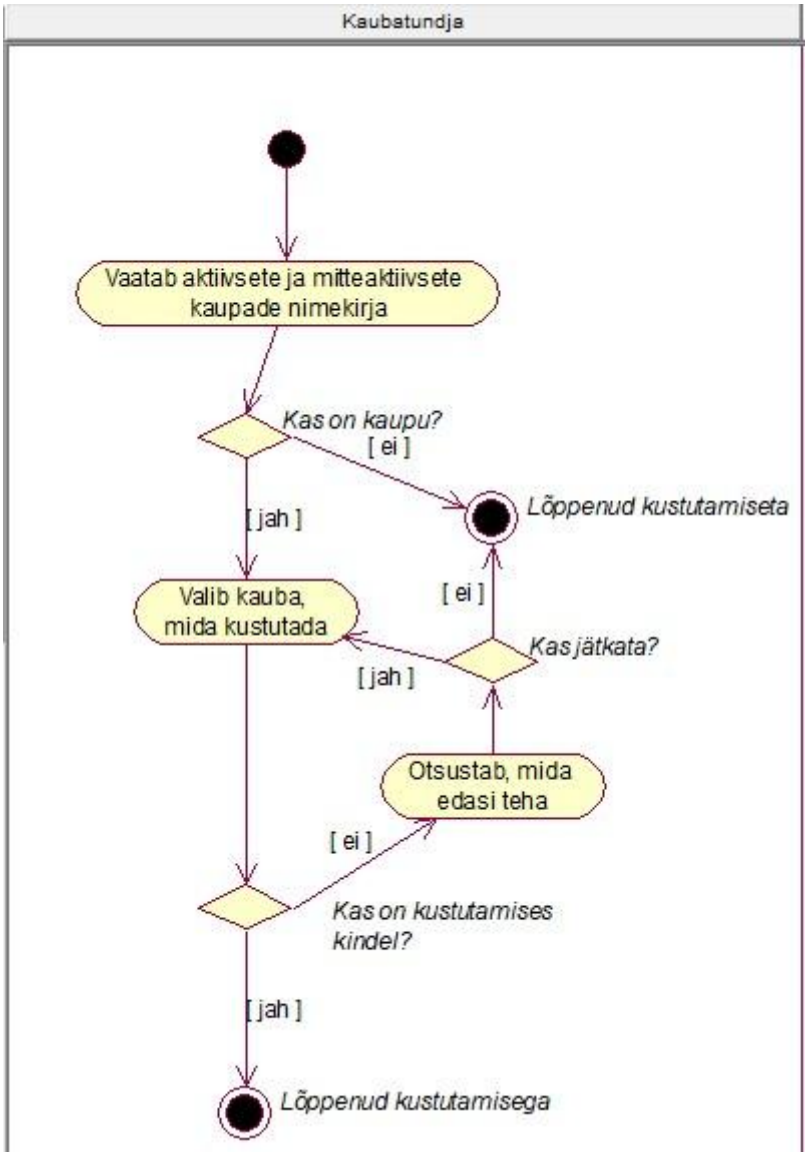
Kui kasutusjuhtude mudelis viidata andmebaasioperatsioonide lepingutele ja andmebaasioperatsioonide lepingute juures kasutusjuhtudele, siis see aitab kontrollida nende mudelite omavahelist kooskõla.

- Igale andmebaasioperatsioonile tuleb viidata vähemalt ühes süsteemi kasutamist kirjeldavas kasutusjuhus.
- Iga süsteemi kasutamist kirjeldav kasutusjuht peab viitama vähemalt ühele andmebaasioperatsioonile.

Andmebaasioperatsioonid on nagu liim, mis seob omavahel ära andmed (kirjeldatakse kontseptuaalses andmemudelis) ja protsessid (kirjeldatakse kasutusjuhtude stsenaariumitena).

Tegevusdiagramm

Tegevusdiagrammide abil saab kirjeldada protsesse. Tegevusdiagrammis saab näidata, mis sammud tuleb protsessi läbi tegemiseks läbi teha, kes neid samme astub ning mis järjekorras need toimuvad. Kuna protsessis võib olla valikukohti ja tsükleid, siis ühe ja sama protsessi läbimiseks võib olla erinevaid sammude jadasid, mis on erinevad stsenaariumid. Tegevusdiagrammide abil saab modelleerida väga erinevaid protsesse – nii selliseid, mis ei hõlma üldse IT vahendite kasutust, üldiseid protsesse, mis hõlmavad erinevaid funktsionaalseid allsüsteeme ning pädevusalasid, kui ka kasutusjuhtude poolt esitatavaid elementaarseid äriprotsesse. Just ühte sellist elementaarset äriprotsessi esitab Joonis 7. Laiendatud formaadis kirja pandud kasutusjuhus oleks selle protsessi kirjeldus esitatud tekstiliselt – põhistsenaariumina ja alternatiivsete stsenaariumitena. Need stsenaariumid kirjeldaksid tegutseja ja süsteemi vahelist suhtlust. Tegevusdiagramm võimaldab selle info esitada visuaalselt. Pange palun tähele, et tegevusdiagrammis ei ole "süsteem" eraldi tegutsejaks ("ujumisrajaks"). Tegevusdiagrammis esitatud tegutseja(d) on kujutatud ka kasutusjuhtude mudelis.



Joonis 7 Kauba lõpetamise tegevusdiagramm, mis väljendab kasutusjuhu "Lõpeta kaup" erinevaid stsenaariume.

CRUD maatriks

Veel üheks mudelite omavahelise kooskõla kontrollimise võimaluseks on CRUD maatriksi loomine, mis võimaldab kontrollida andmete ja protsesside kooskõla.

Kasutusjuhud:

1 – Registreeri kaup
2 – Muuda kaup mitteaktiivseks
3 – Muuda kaup aktiivseks
4 – Kustuta kaup
5 – Muuda kaupa
6 – Vaata kustutamata kaupu
7 – Vaata kaupade detailaruannet

Näide: „U" tähele siin vastab andmebaasioperatsioon **OP2**.

Tabel 2 CRUD maatriks.

Kasutusjuht ---------- Olemitüüp	1	2	3	4	5	6	7
Kaup	C	R**U**	RU	RU	RU	R	R
Kauba_seisundi_liik	R	R	R	R	R	R	R
Pakendi_tüüp	R	R	R	R	R	R	R
Klassifikaator	R	R	R	R	R	R	R

Tabelis 2 esitatav CRUD maatriks näitab, et *kaupade funktsionaalse allsüsteemi* kasutusjuhud haldavad (loovad (C) / loevad (R) / muudavad (U) / kustutavad (D)) kaupade registris olevaid andmeid (antud näite korral andmeid, mis vastavad olemitüübile *Kaup*). Ülejäänud olemitüüpidele vastavaid andmeid ainult loetakse (R). Pange tähele, et *kaupade funktsionaalne allsüsteem* ei halda seisundi klassifikaatori ja pakendi tüübi andmeid – see on *klassifikaatorite funktsionaalse allsüsteemi* vastutus.

Igale C, R, U ja D tähele CRUD maatriksis vastab mingi andmebaasioperatsioon. Küsimus on selles, kas ja milliseid andmebaasioperatsioone otsustatakse projektis lepingu formaadis kirjeldada. Näiteks võidakse otsustada lugemisoperatsioonide kohta lepinguid mitte kirja panna. Põhimõtteliselt saab ka lugemisoperatsioone lepingu formaadis kirjeldada. Selliste operatsioonide lepingute järeltingimused kirjeldaksid, millised andmed on vaja operatsiooni tulemusena leida.

Näiteks kasutusjuhu *Muuda kaup mitteaktiivseks* (2) täitmine algab sellega, et kaubatundja vaatab selliste kaupade nimekirja, mida saab muuta mitteaktiivseks. Selles nimekirjas on andmed aktiivsete kaupade kohta. Nimekirja koostamiseks peab süsteem lugema (R) olemitüüpidele *Kaup*, *Kauba_seisundi_liik*, *Pakendi_tüüp* ja *Klassifikaator* vastavaid andmeid. Seetõttu ongi CRUD maatriksis veerus 2 kõikides ridades R täht. Seejärel valib kaubatundja konkreetse kauba, mida mitteaktiivseks muuta. Süsteem vastab OP2 käivitamisega, mis muudab kaupa, mitte kauba seisundi liiki (vt andmebaasioperatsiooni OP2 lepingut). Seetõttu ongi CRUD maatriksis veerus 2 *Kaup* juures U täht, aga U tähte ei ole *Kauba_seisundi_liik* juures.

Järgnevad mustrid: Põhiolemite hetkeseisundi registreerimine SQL-andmebaasis

Nimi: Põhiolemite hetkeseisundi registreerimine SQL-andmebaasis

Ingliskeelne nimi: Registration of the current state of the main entities in a SQL database

Eelmised mustrid: Põhiolemite hetkeseisundi registreerimise vajadus

Probleem: Milline SQL-andmebaas tuleks loogilise disaini käigus kavandada, et oleks tagatud võimalus säilitada andmebaasis andmeid põhiolemitüüpi kuuluvate olemite (põhiolemite) hetkeseisundi kohta?

Kontekst:

- Detailanalüüsi mudelis on fikseeritud põhiolemite hetkeseisundi registreerimise vajadus.
- Detailanalüüsi kontseptuaalses andemudelis kirjeldatakse põhiolemitüübile vastav seisundi klassifikaator.

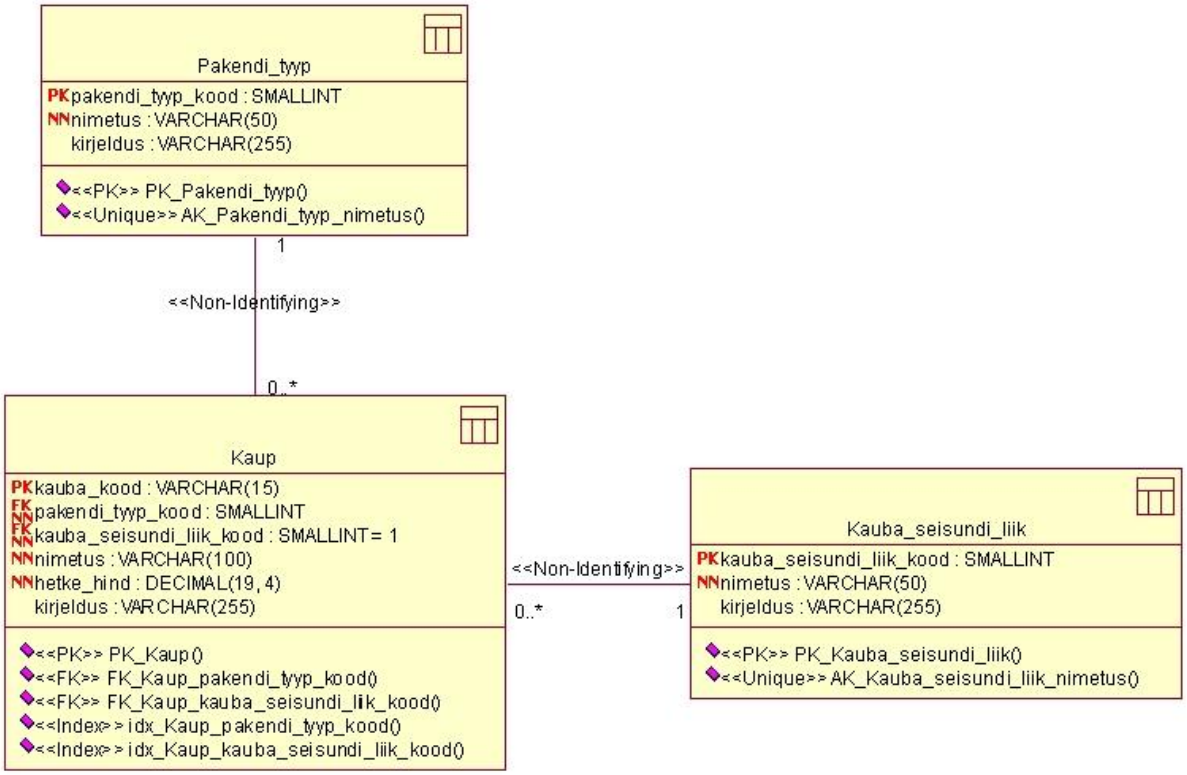
Lahendus: Luua põhiolemitüübi ja selle seisundi klassifikaatori kirjelduste põhjal kaks eraldi baastabelit (tabelit). Lisada põhiolemitüübi ja seisundi klassifikaatori vahelise seosetüübi alusel põhiolemitüübile vastavasse tabelisse välisvõtme veerg ning deklareerida sellele veerule välisvõtme kitsendus. Välisvõtme kitsendusega seotud kompenseerivate tegevuste määrangud on ON DELETE NO ACTION ON UPDATE CASCADE. ON DELETE NO ACTION tagab, et kui keegi üritab kustutada seisundi klassifikaatori väärtust, siis see ei õnnestu, kui selle klassifikaatori väärtusega on seotud vähemalt ühe põhiolemi andmed. ON UPDATE CASCADE tagab, et klassifikaatori koodi muutmisel seisundi klassifikaatori tabelis kantakse muudatus üle tabelisse, milles on põhiolemite andmed.

Juhime tähelepanu, et klassifikaatorite andmete muutmine on vastutusrikas ja suure mõjuga tegevus, mida tehakse vastavate õigustega osapoole poolt. Tuleb tagada, et selline muudatus ei tekita süsteemis vigu (eeldab põhjalikku testimist). Klassifikaatori koodi muutmisel võib lisaks dokumentatsiooni muutmisele olla vajalik muuta vaadete aluseks olevaid päringuid, rakendustes sisalduvaid päringuid, andmete muutmise lauseid nii rakenduses kui ka andmebaasiserveris talletatud rutiinides ja põhiolemitüübi alusel loodud tabelis oleva seisundi klassifikaatorile viitava välisvõtme veeru vaikimisi väärtust.

Deklareerida välisvõtme veerule vaikimisi väärtus, milleks on selle seisundi kood, millesse põhiolem registreerimisel kõige esimesena läheb (see leitakse põhiolemitüübi seisundidiagrammi alusel). Kui seisundidiagrammi kohaselt leidub mitu võimalikku algseisundit, siis sellisel juhul tuleb valida sellise seisundi kood, millesse olem läheb kõige tõenäolisemalt või siis vaikimisi väärtust üldse mitte deklareerida. Seisundi klassifikaatori tabelisse lisatavad andmed leitakse põhiolemitüübi seisundidiagrammi põhjal.

Näide: Joonis 8 esitab lihtsustatud kaupade registri olemi-suhte diagrammi (vt Joonis 2) alusel loodud andmebaasi diagrammi, mis kirjeldab SQL-andmebaasi loogilist disaini. Kontseptuaalses andmemudelis esitati ülatüüp *Klassifikaator* ja selle kaks alamtüüpi – *Pakendi_tüüp* ja *Kauba_seisundi_liik*. Tulenevalt kontseptuaalses andmemudelis üldistuste hulgaga seotud kitsendusest *{Mandatory; Or}*, on loodud tabel iga alamtüübi kohta, kuid ei ole loodud eraldi tabelit ülatüübi (*Klassifikaator*) kohta.

Pange tähele, et tabelis *Kaup* on veerule *kauba_seisundi_liik_kood* deklareeritud vaikimisi väärtus „1“. See väärtus vastab kauba seisundi liigile „Aktiivne“ (vt Tabel 1). Seda väärtust kasutatakse vaikimisi väärtusena, sest kauba seisundidiagrammi kohaselt läheb kaup registreerimisel seisundisse „Aktiivne“ (vt Joonis 3).



Joonis 8 Lihtsustatud kaupade registri loogilist disaini kirjeldav andmebaasi diagramm.

Tabelites *Kaup* ja *Kauba_seisundi_liik* on veerg *kauba_seisundi_liik_kood* tüüpi SMALLINT. Miks mitte INTEGER? Tüüp on nime omav lõplik väärtuste hulk. Veerule tüübi määramine määrab võimalikud väärtused, mis selles veerus üldse võivad olla. Tüübi valikul tuleb mõelda, millised on võimalikud väärtused selles veerus praegu ja tulevikus. Kuna erinevaid seisundeid on väga vähe, siis tuleks tüübi INTEGER asemel (sellesse kuuluvad täisarvulised väärtused -2147483648 kuni +2147483647) kasutada tüüpi SMALLINT (sellesse kuuluvad täisarvulised väärtused -32768 kuni +32767). Lisaks salvestusruumi kokkuhoidmisele annab see andmebaasi skeemiga tutvujale (inimene või tarkvara) lisainfot, milliseid andmeid

plaanitakse hakata selles veerus hoidma. See on nagu koodi tasemel süsteemi dokumenteerimine.

Pange tähele, et klassifikaatorite tabelites (*Pakendi_tyyp* ja *Kauba_seisundi_liik*) ei genereerita ridade lisamisel koode süsteemi poolt automaatselt, vaid sisestatakse vastavate volitustega kasutajate poolt. Miks see on vajalik? Näiteks võimaldab see tabelis *Kaup* deklareerida veerule *kauba_seisundi_liik_kood* vaikimisi väärtuse (kuna on üheselt teada, milline kood vastab igale klassifikaatori väärtusele) ning samuti lihtsustab päringute kirjutamist.

Pange ka tähele, et klassifikaatorite tabelites (*Pakendi_tyyp* ja *Kauba_seisundi_liik*) on veerule *nimetus* deklareeritud unikaalsuse kitsendus. Nõue selle kohta peab väljenduma kontseptuaalses andmemudelis olemitüübi *Klassifikaator* atribuudi *nimetus* definitsioonis.

Veeru *hetke_hind* tüübiks on DECIMAL(19,4), mis on küllaltki levinud lahendus:

https://stackoverflow.com/questions/224462/storing-money-in-a-decimal-column-what-precision-and-scale

MS Accessi korral võib kasutada selle veeru tüübina ka MS Accessi-spetsiifilist tüüpi CURRENCY.

Näide: Leia kaubad, mis on seisundis „Aktiivne“.

Lahendus, kui *kauba_seisundi_liik_kood* väärtused oleks süsteemi-genereeritud (st kood võib ajas muutuda – klassifikaatori väärtuse kustutamisel ja uuesti registreerimisel genereeritakse sellele uus kood; milline kood on klassifikaatori väärtusega seotud saadakse teada alles andmete registreerimisel). Sellisel juhul tuleb teha otsing seisundi nime järgi ja see eeldab ühendamisoperatsiooni läbiviimist.

```
SELECT K.kauba_kood, pakendi_tyyp_kood, K.nimetus, hetke_hind,
K.kirjeldus
FROM Kaup AS K INNER JOIN Kauba_seisundi_liik AS Ks ON
K.kauba_seisundi_liik_kood=Ks.kauba_seisundi_liik_kood
WHERE Ks.nimetus='Aktiivne';
```

Lahendus, kui *kauba_seisundi_liik_kood* väärtused on kasutaja-defineeritud (st on üheselt teada, milline on igale seisundile vastav kood).

```
SELECT kauba_kood, pakendi_tyyp_kood, nimetus, hetke_hind,
K.kirjeldus
FROM Kaup
WHERE kauba_seisundi_liik_kood=1;
```

Lahendused erinevad üksteisest nii kirjapildi lihtsuse kui ka võimaliku jõudluse poolest (ühendamisoperatsiooni läbiviimine vähendab suure tõenäosusega jõudlust).

Tabelid 3 ja 4 esitavad tabelite *Kauba_seisundi_liik* ja *Kaup* näiteväärtused. Tabeli *Kaup* kehandis on üks rida – see esitab andmed kauba kohta, mis on seisundis „Aktiivne“ (kauba_seisundi_liik_kood =1).

Tabel 3 Tabeli Kauba_seisundi_liik näiteväärtus.

seisundi_liik_kood	nimetus	kirjeldus
1	Aktiivne	Kaup jõuab sellesse seisundisse, kui kauplusesse võetakse müügile uus kaup või kui otsustatakse jätkata ajutiselt peatatud kauba müüki.
2	Mitteaktiivne	Kaup jõuab sellesse seisundisse, kui kauba müük otsustatakse ajutiselt peatada.
3	Kustutatud	Kaup jõuab sellesse seisundisse kui kauba müük otsustatakse lõplikult lõpetada.

Tabel 4 Tabeli Kaup näiteväärtus.

kauba_kood	pakendi_tyyp_ kood	kauba_seisundi_ liik_kood	nimetus	hetke_hind	kirjeldus
1244AA56D	1	1	HP Compaq LA2205WG	100	küsitakse sageli

Kui soovitakse muuta konkreetse kauba seisundit, siis tuleb muuta andmeid tabelis *Kaup*. Järgneva lausega muudetakse kauba, mille kood on „1244AA56D“, seisundit nii, et uus seisund on „Mitteaktiivne“.

```
UPDATE Kaup SET kauba_seisundi_liik_kood=2
WHERE kauba_kood='1244AA56D';
```

Tabel 5 esitab tabeli *Kaup* näiteväärtuse selle muudatuse järel.

Tabel 5 Tabeli Kaup näiteväärtus.

kauba_kood	pakendi_tyyp_ kood	kauba_seisundi_ liik_kood	nimetus	hetke_hind	kirjeldus
1244AA56D	1	2	HP Compaq LA2205WG	100	küsitakse sageli

Kuna tabelis *Kaup* on defineeritud välisvõti:

(kauba_seisundi_liik_kood) Viitab Kauba_seisundi_liik (kauba_seisundi_liik_kood),

siis järgmine muudatus ebaõnnestub, sest sellega üritatakse registreerida andmeid, mis on vastuolus viidete terviklikkuse reegliga.

```
UPDATE Kaup SET kauba_seisundi_liik_kood=5
WHERE kauba_kood='1244AA56D';
```

Tabelis *Kauba_seisundi_liik* ei ole registreeritud seisundit, mille kood on 5.

Nimi: Põhiolemite seisundimuudatuste võimaldamine graafilises kasutajaliideses

Ingliskeelne nimi: Enabling state changes of the main entities in the graphical user interface

Eelmised mustrid: Põhiolemite hetkeseisundi registreerimise vajadus

Probleem: Kuidas realiseerida graafilises kasutajaliideses kasutusjuhud, mille sisuks on põhiolemi seisundi muutmine?

Kontekst:

- Detailanalüüsi mudelis on fikseeritud põhiolemite hetkeseisundi registreerimise vajadus.
- Detailanalüüsi kontseptuaalses andemudelis kirjeldatakse põhiolemitüübile vastav seisundi klassifikaator.

Lahendus: Oletame, et meil on vaja realiseerida kasutusjuht, mille sisuks on põhiolemitüüpi OT kuuluvate olemite viimine seisundisse s_x. Oletame, et OT seisundidiagrammi kohaselt saab seisundisse s_x viia põhiolemeid, mis on seisundites $\{s_a,\ldots,s_n\}$. Sellisel juhul tuleb graafilises kasutajaliideses esitada nimekiri põhiolemitüüpi OT kuuluvatest olemitest, mis on seisundites $\{s_a,\ldots,s_n\}$. Iga põhiolemi kohta tuleb esitada selle kõige olulisemad andmed. Nimekirjas tuleb kindlasti esitada andmed, mis võimaldavad inimkasutajatel põhiolemeid sisuliselt üksteisest eristada. Kui hulka $\{s_a,\ldots,s_n\}$ kuulub rohkem kui üks seisund, siis tuleb nimekirjas iga põhiolemi kohta esitada ka selle seisundi lõppkasutajale mõistetav identifikaator. Kui hulka $\{s_a,\ldots,s_n\}$ kuulub täpselt üks seisund, siis pole mõtet iga põhiolemi juures sama seisundi identifikaatorit korrata. Sellisel juhul tuleb seisundi identifikaator esitada vormil nii, et oleks üheselt arusaadav, et see käib kõigi nimekirjas olevate põhiolemite kohta.

Kasutajatel peaks olema võimalik sellest nimekirjast andmeid otsida ja nimekirja sorteerida (sorteerimine aitab samuti andmeid otsida). Iga põhiolemi andmete juures peab olema võimalus anda korraldus viia see olem seisundisse s_x.

Kui OT seisundidiagrammi kohaselt läheb OT tüüpi põhiolem loomise järel ühte kindlasse seisundisse, siis ei tohi OT tüüpi põhiolemit registreerimise vormis olla võimalik esialgset seisundit valida – süsteem peab selle ise määrama. Juhul kui OT tüüpi põhiolem võib loomise järel minna rohkem kui ühte seisundisse, siis tuleb selliste olemite registreerimise vormil pakkuda valikuna välja ainult need seisundid.

Kasutuaja arvutis olev rakendus (MS Accessi näitel)

Näide: Järgnev näide on teostatud töölaua andmebaasisüsteemis MS Access, kuid sarnased üldpõhimõtted on rakendatavad mistahes „paksu kliendi“ tüüpi rakenduse korral, sõltumata realisatsiooni keskkonnast.

Vaatleme näitena kaupluse infosüsteemi. Joonis 6 kohaselt on *kaupade funktsionaalse allsüsteemi* üheks kasutusjuhuks *Muuda kaup mitteaktiivseks*. Graafilises kasutajaliideses võiks selle kasutusjuhu realiseerimiseks olla esitatud nimekirjavorm (vt Joonis 9), kus esitatakse seisundis „Aktiivne“ olevate kaupade andmed. Selle nimekirjavormi aluseks on vaade, mille moodustab järgnev päring.

```
SELECT Kaup.kauba_kood, Pakendi_tyyp.nimetus AS pakend,
Kaup.nimetus, Kaup.hetke_hind FROM Pakendi_tyyp INNER JOIN
Kaup ON Pakendi_tyyp.pakendi_tyyp_kood=Kaup.pakendi_tyyp_kood
WHERE Kaup.kauba_seisundi_liik_kood=1
ORDER BY Kaup.kauba_kood;
```

Aktiivsed kaubad

	kauba kood	pakend	nimetus	hetke väljamüügihind käibemaksuga (EUR)
Muuda mitteaktiivseks	1244AA56D	Pappkast	HP Compaq LA2205WG	80
Muuda mitteaktiivseks	24234LOP	Pappkast	HP Compaq 5000	200

Joonis 9 Nimekirjavorm, mis võimaldab muuta kaupu mitteaktiivseks.

Nimekirjavormis esitatakse kauba kood. Kauba kood on kasutajatele tähendust omav väärtus, mis võimaldab neil kaupu üksteisest eristada. Nagu näete, siis ühendatakse päringus tabel *Kaup* tabeliga *Pakendi_tyyp*, et tulemuses näidata ka pakendi tüübi nimetust. Kuigi pakendi tüübi kood on samuti sisulise tähendusega väärtus, siis pakendi tüübi unikaalne nimetus võib olla lõppkasutajate jaoks lihtsamini arusaadav. Kuna mitteaktiivseks saab muuta vaid seisundis „Aktiivne“ olevaid kaupu, siis on päringus piirang (Kaup.kauba_seisundi_liik_kood=1), mille tulemusena on päringu tulemuses andmed ainult seisundis „Aktiivne“ olevate kaupade kohta. Pange tähele, et selline piirang on võimalik kirjutada päringusse tänu sellele, et on üheselt teada, et *kauba_seisundi_liik_kood* väärtus 1 tähendab alati seisundit „Aktiivne“ (vt Tabel 1).

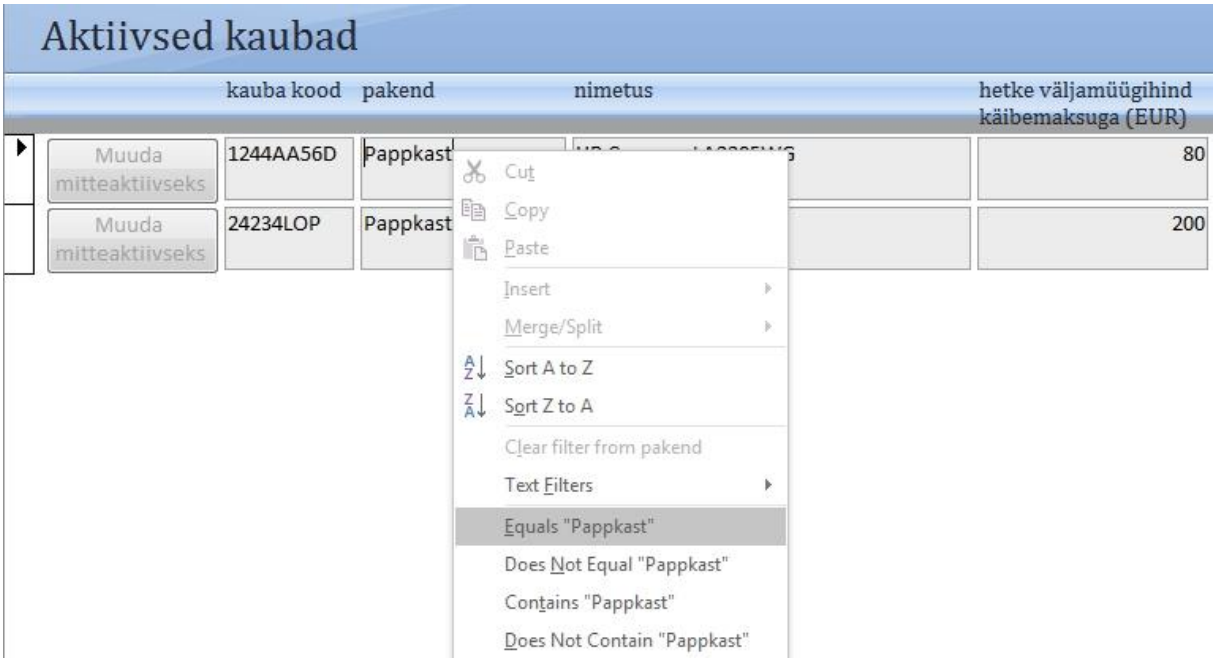
Kuna selles nimekirjavormis esitatavad kaubad on kõik ühes seisundis („Aktiivne“), siis ei esitata nimekirjavormis iga kauba kohta eraldi selle kauba seisundi nimetust. Selle asemel öeldakse vormil oleva teksti abil, et vormil esitatakse aktiivsete kaupade andmed. Juhul kui nimekirjavormis võib olla andmeid erinevates seisundites kaupade kohta, siis tuleb

nimekirjavormis (ja seega ka selle aluseks olevas vaates) esitada iga kauba seisundi nimetus (vt Joonis 10). Seisundi „Aktiivne“ esile tõstmiseks on kasutatud tingimuslikku vormindamist (vt Joonis 10).

Joonis 10 Nimekirjavorm, kus esitatakse kaupade seisundi info.

Kauba mitteaktiivseks muutmiseks tuleb leida sobiv kaup ja vajutada selle juures nupule *Muuda mitteaktiivseks*.

MS Access keskkonnas loodud nimekirjavormide puhul saab lihtsamaks andmete otsimiseks kasutada ka süsteemi sisseehitatud võimalusi (vt Joonis 11). Paraku ei tööta need rakenduse avamisel (tasuta pakutavas) MS Accessi rakenduste käituskeskkonnas MS Access Runtime. Rakenduse selle abil kasutamise korral tuleb rakendusse otsimine (sorteerimine ja filtreerimine) ise sisse ehitada.



Joonis 11 Nimekirjavorm koos sisseehitatud andmete otsimise võimalusega.

Nupule *Muuda mitteaktiivseks* vajutades käivitub järgnev *Visual Basic for Applications* abil kirjutatud sündmusprotseduur. See protseduur on seotud ekraaninupu sündmusega *On Click*.

```
Private Sub cmd_mitteaktiivseks_Click()
'Muutuja deklareerimine
Dim sql As String
'Kasutajalt kinnituse küsimine
If MsgBox("Kas olete kindel, et soovite muuta kauba andmed
mitteaktiivseks?", vbYesNo, "Küsimus") = vbYes Then
'Kui kasutaja on nõus
    'SQL lause koostamine. Pange tähele, et kuna veerg kauba_kood on
tüüpi VARCHAR, siis
    'peab koostatud SQL lauses olema kauba kood ülakomade vahel.
    'Kui veerg kauba_kood oleks täisarvu tüüpi, siis neid ülakomasid
ei oleks vaja.
    'Me.kauba_kood on viide nimekirjavormi reas väljas nimega
kauba_kood olevale väärtusele.
    sql = "UPDATE Kaup SET kauba_seisundi_liik_kood=2 WHERE
kauba_kood ='" & Me.kauba_kood & "'"
    'Hoiatuste väljalülitamine
    DoCmd.SetWarnings False
    'Koostatud SQL lause käivitamine
    DoCmd.RunSQL sql
    'Hoiatuste sisselülitamine
    DoCmd.SetWarnings True
    'Nimekirjavormi värskendamine
    Me.Requery
    Me.Refresh
End If
End Sub
```

Rõhutame, et see sündmusprotseduur realiseerib andmebaasioperatsiooni **OP2**, mille leping on esitatud mustris *Põhiolemite hetkeseisundi registreerimise vajadus*. Pange tähele, et sündmusprotseduur ei kontrolli antud juhul eeltingimusi ega invariante. See on süsteemi teiste osade vastutus ning võimalus neid operatsiooni alusel loodud rutiinis mitte kontrollida ongi üks lepingprojekteerimist kasutades loodud süsteemi võludest.

Antud juhul saavutatakse see, et järeltingimuste täidetuseni jõutakse vaid siis, kui eeltingimused on täidetud tänu sellele, et:

- andmebaasis on deklareeritud välisvõtme kitsendused. Tänu sellele, ei saa näiteks kaupa siduda seisundi liigiga, mille kood on 5.
- andmebaasis on jõustatud reegel, et kaup saab minna seisundisse koodiga 2 vaid siis, kui kaup on seisundis koodiga 1. Selle reegli jõustamise kohta lugege mustrist Põhiolemite teatavate seisundimuudatuste keelamine SQL-andmebaasis.

```
OP2 Muuda kaup mitteaktiivseks (

      p_kauba_kood
)
Eeltingimused:
• Kaup eksemplar k (kauba_kood= p_kauba_kood) on registreeritud
• k on seotud Kauba_seisundi_liik eksemplariga ksl_vana (kood=1
  ("Aktiivne"))
• Kauba_seisundi_liik eksemplar ksl_uus (kood=2 ("Mitteaktiivne"))
  on registreeritud
Järeltingimused:
--Kustuta seoseid
• k ja ksl_vana seos on kustutatud
--Loo seoseid
• k ja ksl_uus seos on registreeritud
Kasutus kasutusjuhtude poolt: Muuda kaup mitteaktiivseks
```

Programm küsib kasutajalt, kas ta on kindel, et soovib muuta kauba andmed mitteaktiivseks. Kui kasutaja vastab jaatavalt, siis koostab programm seisundi muutmiseks UPDATE lause ja käivitab selle lause. Koostatud UPDATE lause antakse muutuja *sql* väärtuseks.

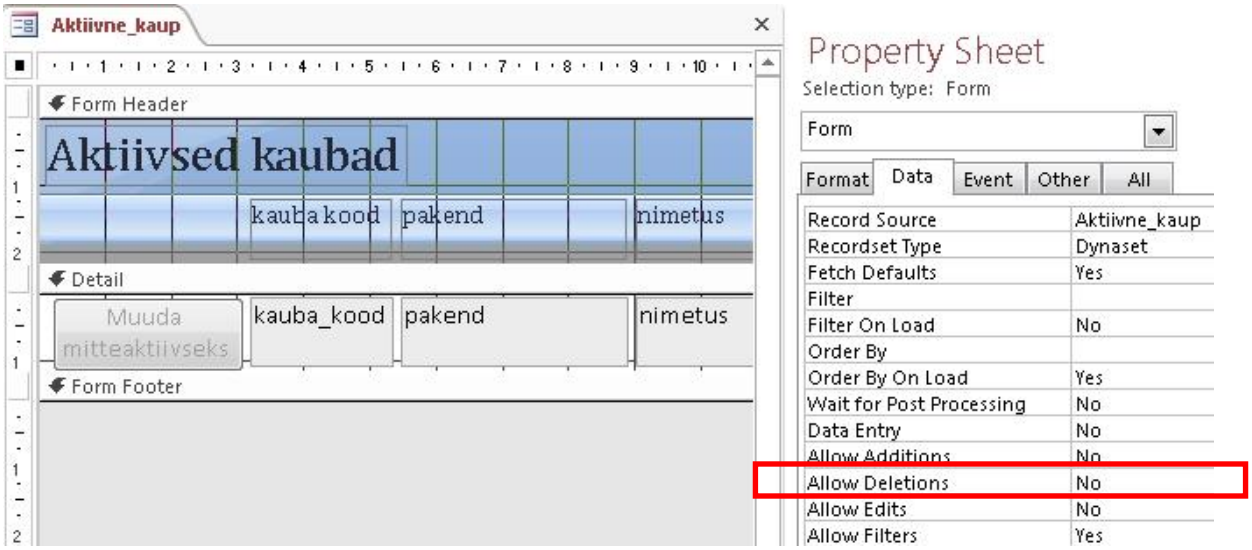
Võib tekkida küsimus, miks mitte lubada muuta seisundi andmeid nagu ülejäänud andmeid (vt Joonis 12). Kui lubada kauba uut seisundit Joonis 12 esitatud viisil liitboksist valida, siis saaks andmete haldaja muuta kauba seisundit suvalisest seisundist suvalisse seisundisse. See aga ei ole kooskõlas põhiolemitüübi *Kaup* seisundidiagrammiga (vt Joonis 3). Isegi kui realiseerida rakenduses või andmebaasis seisundi üleminekute kontroll (ebakorrektsed üleminekud lükatakse süsteemi poolt tagasi), siis Joonis 12 esitatud vormis olevad andmed ei anna ikkagi kasutajale infot, millised seisundi üleminekud on lubatud ja millised mitte. Teiste sõnadega, leidub andmetega seotud kitsendus, millest süsteemi kasutaja pole teadlik. Kasutaja, kes pole tööprotsessiga veel täielikult harjunud või kasutab süsteemi uuesti peale vaheaega, peab töötama katse-eksituse meetodil. Seega on Joonis 12 esitatud lahendus halb lahendus.

Joonis 12 Seisundi andmete haldamise halb lahendus.

Uue kauba registreerimise ekraanivormis ei ole samuti seisundi valimise võimalust (vt Joonis 13), sest kaup läheb loomisel ühte kindlasse seisundisse – „Aktiivne“ (vt Joonis 3). Kuna seisundidiagrammi kohaselt puudub kasutajatel võimalus kauba algseisundit valida, siis ei peaks graafiline kasutajaliides ka sellise valiku tegemise võimalust välja pakkuma (ja seeläbi kasutajaid eksitama).

Joonis 13 Uue kauba registreerimise vorm.

Seisundidiagrammi kohaselt ei kustutata kauba andmeid kunagi füüsiliselt andmebaasist. Kauba kustutamisel muudetakse selle seisundit, kuid kauba andmed jäävad andmebaasi. Seega tuleb MS Accessi rakenduses kõigis kauba andmete haldamise vormides määrata omaduse *Allow Deletions* väärtuseks *No* (vt Joonis 14).



Joonis 14 Kauba andmete kustutamise keelamine rakenduses.

Veebirakendus (Oracle APEX näitel)

Näide: Järgnev näide on teostatud serveri-andmebaasisüsteemis Oracle Enterprise Edition (11g, Release 2) ning veebirakenduste loomiseks mõeldud kiirprogrammeerimise vahendis Oracle Application Express (4.1) (Oracle APEX), kuid lahenduse üldpõhimõtted on rakendatavad mistahes veebirakenduse korral sõltumata realisatsiooni keskkonnast (sealhulgas toimib see ka Oracle andmebaasisüsteemi ning Oracle APEX hilisemate versioonide puhul).

Vaatleme näitena kaupluse infosüsteemi.

Loodud programmi saab vaadata aadressilt (väljastpool ülikooli võrku looge eelnevalt FortiClient VPN ühendus):

http://apex.ttu.ee:8000/apex/f?p=KAUBAD_KAUBATUNDJA

Selle lahenduse realisatsiooni detaile selgitab artikkel:
https://stackoverflow.com/questions/9367759/apex-tabular-form-with-row-selector-how-to-set-value-of-column-in-selected-ro

Joonis 6 kohaselt on kaupade funktsionaalse allsüsteemi üheks kasutusjuhuks *Muuda kaup mitteaktiivseks*. Graafilises kasutajaliideses võiks selle kasutusjuhu realiseerimiseks olla esitatud leht (vt Joonis 15), kus esitatakse seisundis „Aktiivne“ olevate kaupade andmed. Sellel lehel esitatavate andmete leidmiseks kasutatakse vaadet.

```
CREATE OR REPLACE VIEW Aktiivne_kaup AS
SELECT Kaup.kauba_kood, Pakendi_tyyp.nimetus AS pakend,
Kaup.nimetus, Kaup.hetke_hind
FROM Pakendi_tyyp INNER JOIN Kaup ON Pakendi_tyyp.pakendi_tyyp_kood
= Kaup.pakendi_tyyp_kood
WHERE Kaup.kauba_seisundi_liik_kood=1
ORDER BY Kaup.kauba_kood WITH CHECK OPTION;
```

Joonis 15 Leht, mis võimaldab muuta kaupu mitteaktiivseks.

Kaupade mitteaktiivseks muutmiseks tuleb märgistada üks või mitu kaupa ja vajutada nupule *Muuda mitteaktiivseks*. Andmebaasis on loodud pakett, mis sisaldab kauba seisundimuudatuste registreerimiseks mõeldud protseduure. Näiteks protseduur *pr_muuda_kaup_mitteaktiivseks* realiseerib andmebaasioperatsiooni **OP2**, mille leping on esitatud mustris *Põhiolemite hetkeseisundi registreerimise vajadus*.

```
OP2 Muuda kaup mitteaktiivseks (
      p_kauba_kood
)
Eeltingimused:
• Kaup eksemplar k (kauba_kood= p_kauba_kood) on registreeritud
• k on seotud Kauba_seisundi_liik eksemplariga ksl_vana (kood=1
  ("Aktiivne"))
• Kauba_seisundi_liik eksemplar ksl_uus (kood=2 ("Mitteaktiivne"))
  on registreeritud
Järeltingimused:
--Kustuta seoseid
• k ja ksl_vana seos on kustutatud
--Loo seoseid
• k ja ksl_uus seos on registreeritud
Kasutus kasutusjuhtude poolt: Muuda kaup mitteaktiivseks
```

```
CREATE OR REPLACE PACKAGE kauba_seisundimuudatused AS
PROCEDURE pr_muuda_kaup_mitteaktiivseks(p_kauba_kood IN
Kaup.kauba_kood%TYPE);
PROCEDURE pr_muuda_kaup_aktiivseks(p_kauba_kood IN
Kaup.kauba_kood%TYPE);
PROCEDURE pr_kustuta_kaup(p_kauba_kood IN Kaup.kauba_kood%TYPE);
END;
/

CREATE OR REPLACE PACKAGE BODY kauba_seisundimuudatused AS
PROCEDURE pr_muuda_kaup_mitteaktiivseks(p_kauba_kood IN
Kaup.kauba_kood%TYPE) IS
/*Protseduur realiseerib andmebaasioperatsiooni OP2. Selle oodatav
sisend on kauba kood ning see määrab kauba seisundiks
'mitteaktiivne'.*/
BEGIN
     UPDATE Kaup SET kauba_seisundi_liik_kood=2 WHERE
kauba_kood=p_kauba_kood;
END;

PROCEDURE pr_muuda_kaup_aktiivseks(p_kauba_kood IN
Kaup.kauba_kood%TYPE) IS
/*Protseduur realiseerib andmebaasioperatsiooni OP3. Selle oodatav
sisend on kauba kood ning see määrab kauba seisundiks 'aktiivne'.*/
BEGIN
     UPDATE Kaup SET kauba_seisundi_liik_kood=1 WHERE
kauba_kood=p_kauba_kood;
END;

PROCEDURE pr_kustuta_kaup(p_kauba_kood IN Kaup.kauba_kood%TYPE) IS
/*Protseduur realiseerib andmebaasioperatsiooni OP4. Selle oodatav
sisend on kauba kood ning see määrab kauba seisundiks
'kustutatud'.*/
BEGIN
     UPDATE Kaup SET kauba_seisundi_liik_kood=3 WHERE
kauba_kood=p_kauba_kood;
END;

END kauba_seisundimuudatused;
/
```

Pange tähele, et koodi edaspidise haldamise ning edasiarendamise lihtsustamiseks on kasulik seda kommenteerida.

Joonis 16 esitab kaupade mitteaktiivseks muutmise lehe Oracle APEX rakenduse disaineri vaates. Seal on võimalik muuta lehe ning sellel olevate elementide väljanägemist ja käitumist.

Joonis 16 tähistab **1** tabelivormi kirjeldust. Tabelivormi aluseks on päring vaate *Aktiivne_kaup* põhjal. Tabelivorm loodi algselt (baas)tabeli *Kaup* põhjal ning seejärel asendati tabelivormi aluseks olev päring päringuga vaate põhjal. Samuti muudeti tabelivormi väljade omadusi nii, et seal ei saa otse andmeid muuta, vaid saab ainult andmeid vaadata (välja omaduse *Display As* väärtuseks on *Display as Text (saves state)*).

```
select
"KAUBA_KOOD",
"PAKEND",
"NIMETUS",
"HETKE_HIND"
from Aktiivne_kaup
```

Joonis 16 tähistab **2** ekraaninupu *Muuda mitteaktiivseks* kirjeldust.

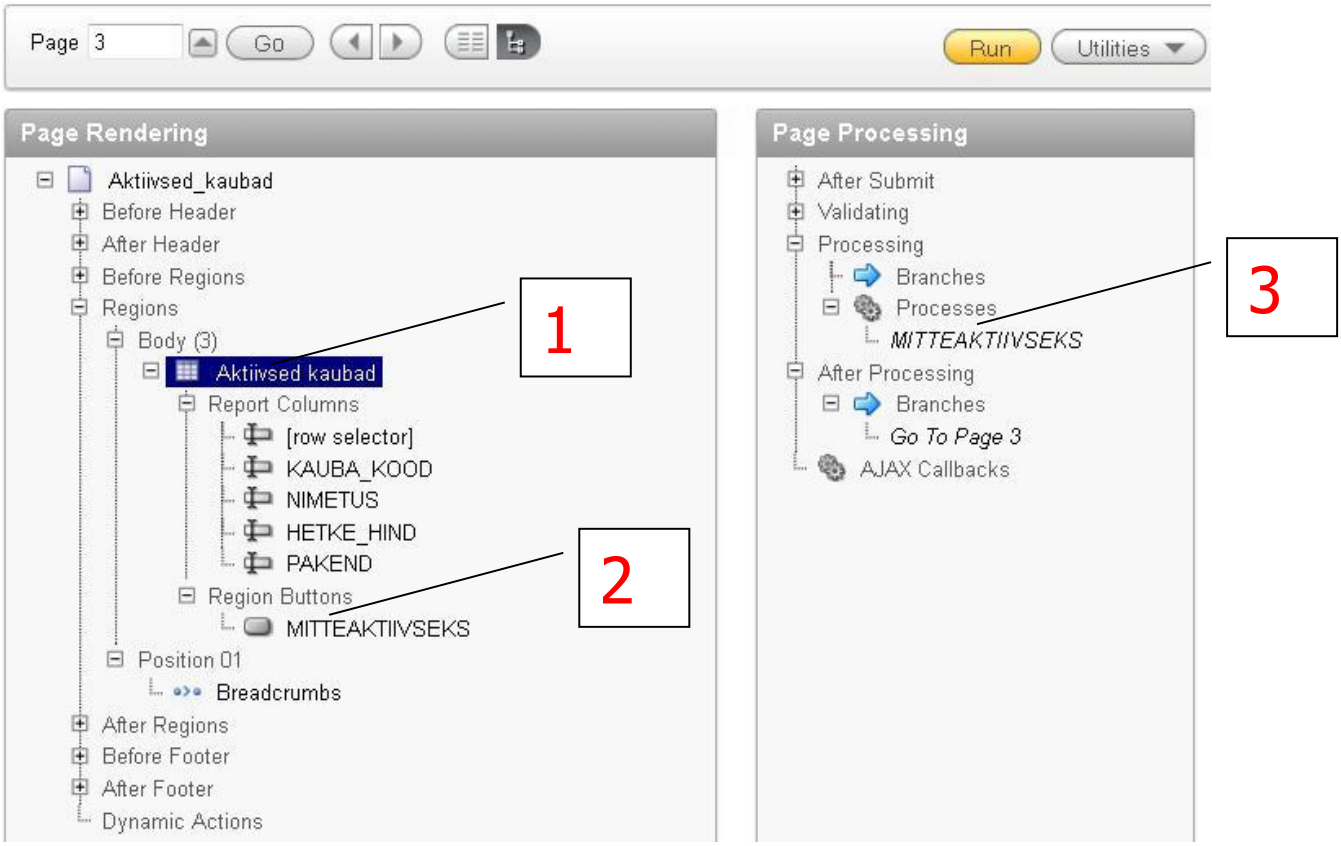
Joonis 16 tähistab **3** nupule *Muuda mitteaktiivseks* vajutamisel käivituva protsessi kirjeldust. Protsess on realiseeritud anonüümse (nimetu) PL/SQL plokina. Protsessi idee seisneb selles, et kui kasutaja märgistab read ja vajutab nupule, siis leiab protsessiga seotud PL/SQL plokk märgistatud ridades olevad kaupade koodid ja kutsub iga märgistatud rea kohta välja paketis *kauba_seisundimuudatused* oleva protseduuri *pr_muuda_kaup_mitteaktiivseks*, kusjuures igal väljakutsel on argumendiks konkreetse kauba kood.

Tabelivormi (vt 1 Joonis 16) esimeses veerus on rea number ja teises veerus kauba kood.

apex_application.g_f01 – massiiv, milles on kasutaja poolt märgistatud ridade numbrid.

apex_application.g_f02 – massiiv, milles on märgistatud ridades olevad kaupade koodid.

```
DECLARE
   v_rowno NUMBER;
BEGIN
   for i in 1..apex_application.g_f01.count
   loop
      v_rowno := apex_application.g_f01(i);
kauba_seisundimuudatused.pr_muuda_kaup_mitteaktiivseks(apex_applicat
ion.g_f02(v_rowno));
   end loop;
END;
```



Joonis 16 Kaupade mitteaktiivseks muutmise lehe vaade rakenduse disainerile.

Joonis 17 ja Joonis 18 esitatakse kaupade mitteaktiivseks muutmise protsessi realisatsioon Oracle APEX keskkonnas. Joonis 18 näitab, et protsess käivitub ainult nupule *Muuda mitteaktiivseks* vajutamise tulemusena.

Name

Page: **3 Aktiivsed_kaubad**
* Name MITTEAKTIIVSEKS
Type: **PL/SQL anonymous block**

Process Point

* Sequence 10
Process Point On Submit - After Computations and Validations
Run Process Once Per Page Visit (default)

Source

* Process [Download Source]

```
DECLARE
   v_rowno NUMBER;
BEGIN
   for i in 1..apex_application.g_f01.count
   loop
      v_rowno := apex_application.g_f01(i);
      kauba_seisundimuudatused.pr_muuda_kaup_mitteaktiivseks(apex_application.g_f02(v_rowno));
   end loop;
END;
```

Joonis 17 Kaupade mitteaktiivseks muutmise protsessi realisatsioon (1).

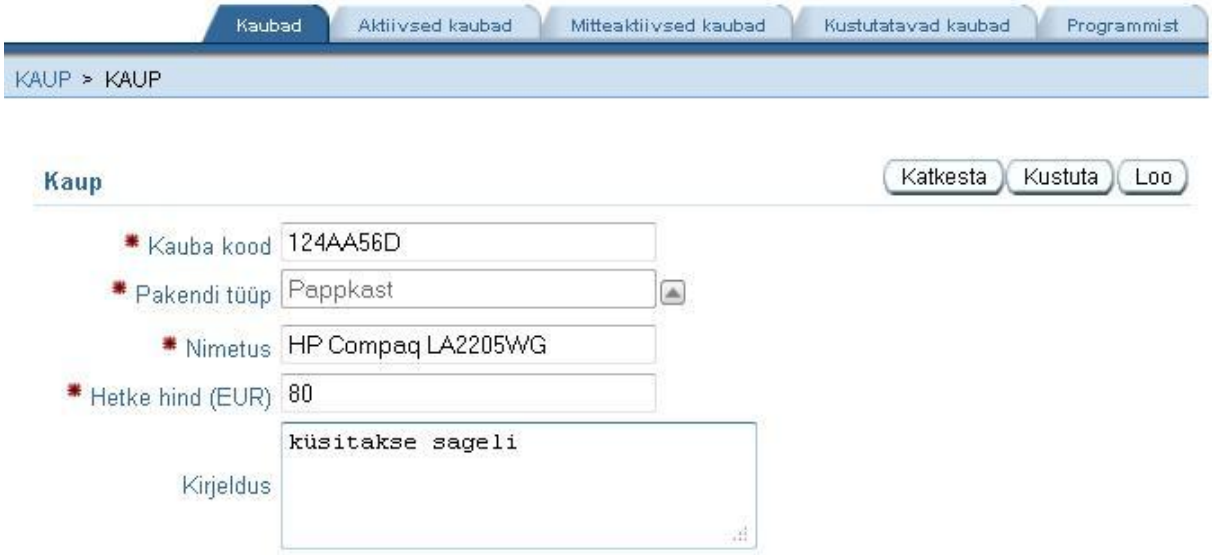
Conditions

When Button Pressed MITTEAKTIIVSEKS (Muuda mitteaktiivseks)

Condition Type
- Select Condition Type -
[PL/SQL] [item / column=value] [item / column not null] [item / column null] [request=e1] [page in] [page not in] [exists] [never] [none]

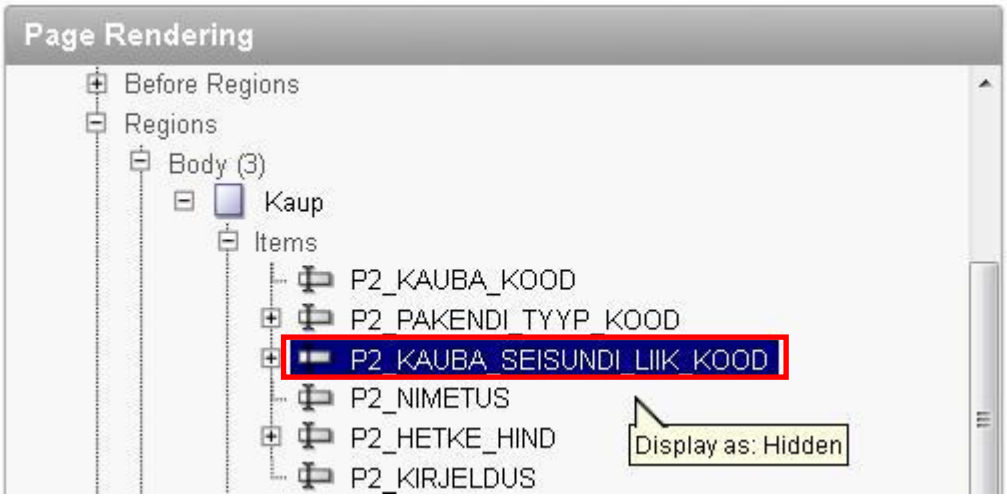
Joonis 18 Kaupade mitteaktiivseks muutmise protsessi realisatsioon (2).

Uue kauba registreerimise lehel ei ole samuti seisundi valimise võimalust (vt Joonis 19), sest kaup läheb loomisel ühte kindlasse seisundisse – „Aktiivne“ (vt Joonis 3). Kuna seisundidiagrammi kohaselt puudub kasutajatel võimalus kauba algseisundit valida, siis ei peaks graafiline kasutajaliides ka sellise valiku tegemise võimalust välja pakkuma (ja seeläbi kasutajaid eksitama).

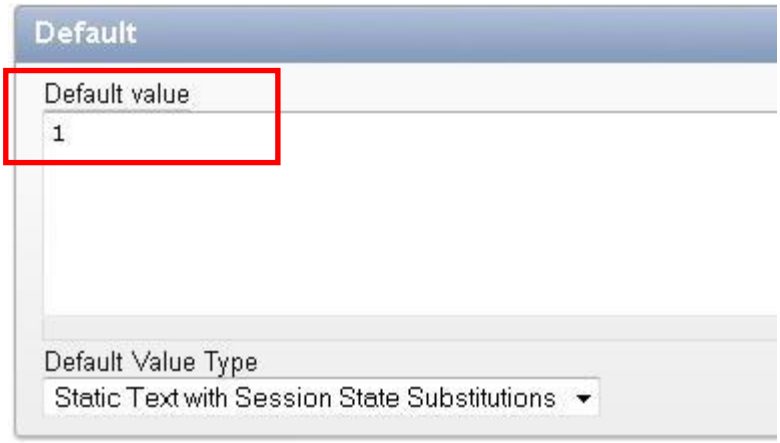


Joonis 19 Uue kauba registreerimise leht.

Uue kauba registreerimise lehel peab väli P2_KAUBA_SEISUNDI_LIIK_KOOD olema peidetud (*hidden*) (vt Joonis 20) ja väljale peab olema defineeritud vaikimisi väärtus (vt Joonis 21) – tabeli veerule vaikimisi väärtuse defineerimisest kahjuks ei piisa.



Joonis 20 Uue kauba registreerimise lehe realisatsioon



Joonis 21 Välja Kaubas_seisundi_liik_kood kirjeldus.

Kontseptuaalses andmemudelis kirjeldatakse muuhulgas olemitüüpide ja atribuutide tähendus. Olemitüüpide ja atribuutide alusel luuakse teisenduste tulemusena SQL-andmebaasi tabelid ja veerud. Tabelite veergudes olevaid andmeid esitatakse kasutajaliideses vormide/veebilehtede kaudu kasutajatele. Kasutajaliidese kasutaja (lõppkasutaja) peab saama aru nende andmete tähendusest, millega ka ta töötab. Seega peavad olemitüüpide aja atribuutide definitsioonides esitatav info jõudma kasutajaliidesesse, kas kasutajaliidese elementide nähtavate nimedena või abiinfona. Oracle APEX pakub käepärase võimaluse väljaga abiinfo sidumiseks (vt Joonis 22). Joonis 23 illustreerib, kuidas tulemus kasutajaliideses välja paistab.

Joonis 22 Väljaga abiinfo sidumine.

Joonis 23 Väljaga seotud abiinfo vaatamine.

Oracle Enterprise Edition andmebaasisüsteem on näide serveri andmebaasisüsteemist. Selliseid süsteeme kasutatakse andmebaaside loomiseks, kus korraga võib samu andmeid soovida lugeda ja muuta mitu kasutajat. **Pakutud protseduuridel ja veebilehel on nõrkus.** Selles dokumendis näidatakse edaspidi, kuidas kontrollida seisundimuudatuste vastavust seisundidiagrammis toodud reeglitele. Kuid kõigele vaatamata on võimalik järgmine stsenaarium.

Aeg	Kasutaja: Juku	Kasutaja: Juhan
t1	Avab kustutatavate kaupade lehe. Süsteem loeb selle moodustamiseks hetkel aktiivsete ja mitteaktiivsete	Avab aktiivsete kaupade lehe. Süsteem loeb selle moodustamiseks hetkel aktiivsete kaupade andmed.

Aeg	Kasutaja: Juku	Kasutaja: Juhan
	kaupade andmed.	
t2		Muudab aktiivse kauba *k* mitteaktiivseks. Süsteem määrab *k* seisundi väärtuseks 2.
t3	Kuna Juku pole vahepeal lehte värskendanud, siis näeb ta endiselt, et *k* on aktiivne. Ta annab korralduse *k* kustutada. Süsteem määrab *k* seisundi väärtuseks 3.	

Seisundidiagrammi seisukohalt on kõik korrektne – ka mitteaktiivset kaupa saab lõpetada.

- Mis siis kui Juku ei oleks mitteaktiivset kaupa lõpetada soovinud?
- Mis siis kui Jukul on õigus lõpetada aktiivset kaupa, kuid pole õigust lõpetada mitteaktiivset kaupa?
- Kui aktiivse ja mitteaktiivse kauba lõpetamine tingib süsteemis erinevaid tegevusi, siis on süsteemi reaktsioon kauba lõpetamisele Juku jaoks üllatuseks.
- Kui süsteem logib kauba seisundimuudatusi, siis pakub hilisem logi uurimine Juku jaoks üllatuse.

Näites toodud protseduurid eelistasid koodi lihtsust tarkvara igas olukorras korrektsele toimimisele. See on valik, millel on oma hind. Kui see hind tundub liiga kõrge, tuleb protseduuride ja rakenduse loogikat täiendada. Lugege näiteks:

https://www.wegobeyond.co.uk/optimistic-row-locking-in-oracle-application-express-apex/

Tõmmake paralleele andmebaasis kitsenduste jõustamisega. Ka seal võib eelistada lihtsust (kitsenduste täidetust ei kontrollita) andmete põhjalikumale kontrollimisele. Väga võimalik, et hilisema andmete kasutamise käigus tuleb selle eest hakata lõivu maksma, st lihtsustamine tähendab *tehnilise võla* võtmist, mida hiljem tuleb koos intressidega tagasi maksta või suure võlakoorma all ägav süsteem üldse ära visata e pankrotti lasta.

Nimi: Põhiolemite teatavate seisundimuudatuste keelamine SQL-andmebaasis

Ingliskeelne nimi: Prohibiting certain state changes of the main entities in a SQL database

Eelmised mustrid: Põhiolemite hetkeseisundi registreerimine SQL-andmebaasis

Probleem: Kuidas keelata SQL-andmebaasi tasemel sellised põhiolemite seisundimuudatused, mis ei ole kooskõlas vastava põhiolemitüübi seisundidiagrammiga?

Kontekst:

- Detailanalüüsi mudelis on fikseeritud põhiolemite hetkeseisundi registreerimise vajadus.
- Detailanalüüsi kontseptuaalses andemudelis kirjeldatakse põhiolemitüübile vastav seisundi klassifikaator.
- Andmebaas realiseeritakse SQL-andmebaasina.
- Andmebaasi kasutab üks või rohkem rakendust, mille kasutajatel on vaja muuda põhiolemite seisundeid.

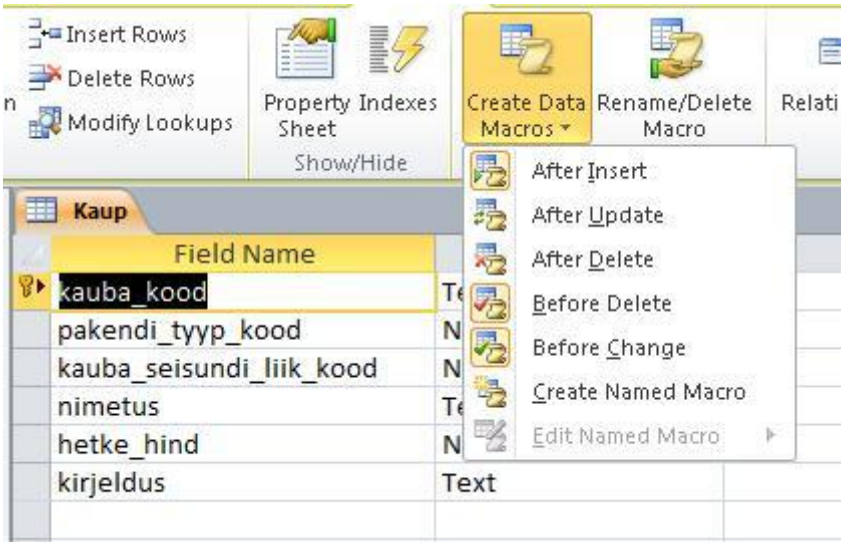
Lahendus: Jõustada SQL-andmebaasis kitsendused, mis ei lase registreerida seisundidiagrammiga mitte kooskõlas olevate seisundimuudatuste tulemusi. Isegi kui selline kontroll on mingil viisil teostatud rakenduses, pakub taoliste kitsenduste jõustamine andmebaasis teise kaitseliini ebakorrektsete andmete registreerimise vastu. Taoline kontroll toimub sõltumata sellest, millise programmi kaudu või milliste õigustega lõppkasutaja poolt andmebaasis andmeid muudetakse. Näiteks kui andmebaasi kasutava rakenduse looja on unustanud sellise kontrolli rakenduses realiseerida, siis tagab andmebaasi tasemel toimuv kontroll, et ebakorrektsed andmed ei jõua andmebaasi. Rakenduste loojad saavad keskenduda muule kui keerukate andmetega seotud kitsenduste jõustamine. Samuti pole sellisel juhul vaja seisundite muutmiseks mõeldud andmebaasiserveris talletatud rutiinides realiseerida eeltingimuste kontrolli, sest andmebaasisüsteem juba kontrollib neid.

MS Access (andmemakrod)

Näide: Järgnev näide on teostatud töölaua andmebaasisüsteemis MS Access (2010) ning toimib ka MS Access hilisemate versioonide puhul. Selliste kitsenduste jõustamiseks tuleb luua andmemakrod, mis reageerivad põhiolemitüübile vastavas tabelis toimunud andmete lisamise, muutmise ning vajadusel ka kustutamise sündmustele. Andmemakro on sarnane SQL-andmebaasi trigerile. Andmemakrosid on MS Accessis võimalik luua alates 2010. aasta versioonist. **NB!** Kui avada andmemakrosid sisaldav andmebaas MS Access 2007 abil, siis ei saa tabelites, millega on seotud üks või mitu andmemakrot, andmeid lisada/muuta/kustutada.

Vaatleme näitena kaupluse infosüsteemi. Kaupluse infosüsteemi SQL-andmebaasis on loodud tabelid *Kaup* ja *Kauba_seisundi_liik* (seisundi klassifikaatori tabel).

Joonis 24 näitab, kuidas siduda tabeliga *Kaup* uus andmemakro.



Joonis 24 Andmemakrode loomine tabelile Kaup.

Tabelile *Kaup* on loodud andmemakro, mis käivitub peale rea lisamist tabelisse (*After Insert*) (vt Joonis 25). Selle makro abil kontrollitakse, et kaup luuakse õiges seisundis. Kui lisatava kauba seisundi liigi kood ei ole 1, siis kustutab süsteem lisatud rea. Selline lahendus valiti, kuna tabelisse rea lisamise järel käivituva andmemakro korral ei saa *RaiseError* käsuga lisamist katkestada ning pole võimalik luua andmemakrot, mis käivitub enne rea lisamist (*Before Insert*).

Joonis 25 After Insert andmemakro, mis on seotud tabeliga Kaup.

Tabelile *Kaup* on loodud andmemakro, mis käivitub enne reas andmete muutmist (*Before Change*) (vt Joonis 26). Tegevused viiakse läbi vaid juhul, kui muudetakse andmeid veerus *kauba_seisundi_liik_kood*. *Updated* on MS Accessis süsteemi-defineeritud funktsioon.

Andmete muutmisel käivituvas makros saab viidata reas enne muutmist ja peale muutmist olevatele väärtustele.

Näiteks avaldises *[Old].[kauba_seisundi_liik_kood]* =2 viidatakse *kauba_seisundi_liik_kood* muutmise eelsele (vanale) väärtusele tabeli *Kaup* muudetavas reas ja avaldises *[kauba_seisundi_liik_kood]=2* viidatakse *kauba_seisundi_liik_kood* muutmise järgsele (uuele) väärtusele tabeli *Kaup* muudetavas reas.

Before Change andmemakros tühistab *RaiseError* käsk makro käivitanud andmemuudatuse. See võimaldab ebakorrektsed andmemuudatused tagasi lükata.

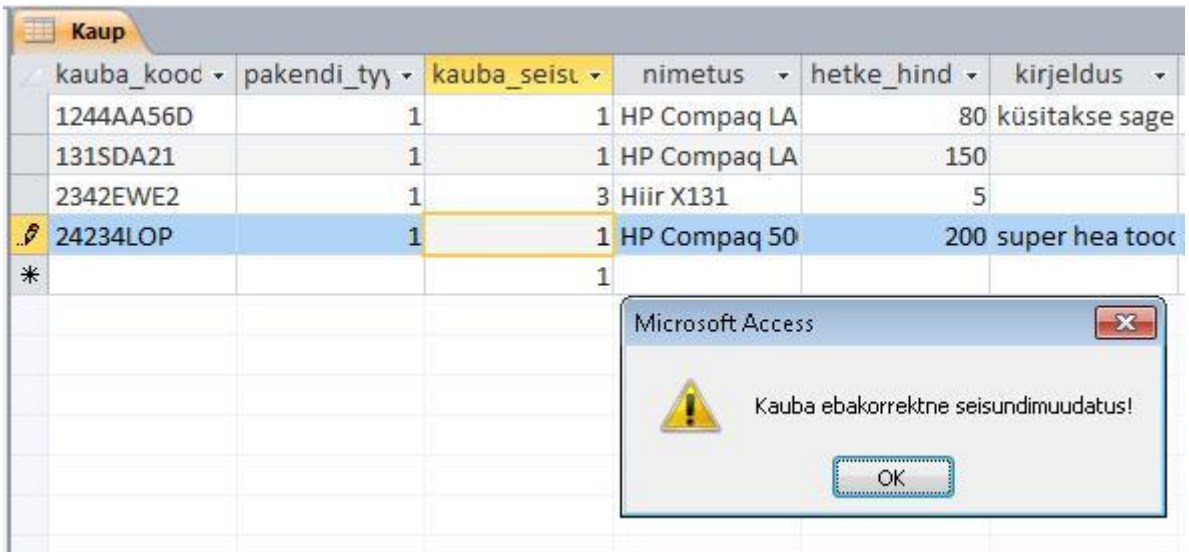
```
Kaup : Before Change :

If Updated("kauba_seisundi_liik_kood") Then
  /* Tegevused viiakse läbi vaid juhul, kui muudetakse andmeid veerus kauba_seisundi_liik_kood.
     Updated on MS Accessis süsteemi-defineeritud funktsioon.
  If Not ([Old].[kauba_seisundi_liik_kood]=1 And [kauba_seisundi_liik_kood]=2) Then
    /* Andmete muutmisel käivituvas makros saab viidata reas enne muutmist ja peale muutmist olevatele väärtustele.

       Näiteks avaldises [Old].[kauba_seisundi_liik_kood] =2 viidatakse kauba_seisundi_liik_kood
       muutmise eelsele (vanale) väärtusele tabeli Kaup muudetavas reas ja
       avaldises [kauba_seisundi_liik_kood]=2 viidatakse kauba_seisundi_liik_kood
       muutmise järgsele (uuele) väärtusele tabeli Kaup muudetavas reas.

    If Not ([Old].[kauba_seisundi_liik_kood]=2 And [kauba_seisundi_liik_kood]=1) Then

      If Not (([Old].[kauba_seisundi_liik_kood]=1 Or [Old].[kauba_seisundi_liik_kood]=2) And [kauba_seisundi_liik_kood]=3) Then
        RaiseError (20003; Kauba ebakorrektne seisundimuudatus!)
        /* Before Change andmete makros tühistab RaiseError käsk makro käivitanud andmemuudatuse.
           See võimaldab ebakorrektsed andmemuudatused tagasi lükata.
      End If
    End If
  End If
End If
Add New Action
```

Joonis 26 Before Change andmemakro, mis on seotud tabeliga Kaup.

Tabelile *Kaup* on loodud andmemakro, mis käivitub enne rea kustutamist (*Before Delete*) (vt Joonis 27). *Before Delete* andmemakros tühistab *RaiseError* käsk makro käivitanud andmemuudatuse. See võimaldab ebakorrektsed andmemuudatused tagasi lükata.

Joonis 27 Before Delete andmemakro, mis on seotud tabeliga Kaup.

Joonis 28 illustreerib, mis juhtub, kui kasutaja üritab ebakorrektset seisundimuudatust. Antud juhul üritati seisundis „Kustutatud“ oleva kauba seisundiks panna „Aktiivne“. Seisundimuudatus ebaõnnestub ja süsteem kuvab vastava teate.



Joonis 28 Ebakorrektse seisundimuudatuse tulemus.

Oracle Enterprise Edition (trigerid)

Näide: Järgnev näide on teostatud serveri-andmebaasisüsteemis Oracle Enterprise Edition (11g, Release 2). Selliste kitsenduste jõustamiseks tuleb luua trigerid, mis reageerivad põhiolemitüübile vastavas tabelis toimunud andmete lisamise, muutmise ning vajadusel ka kustutamise sündmustele.

Vaatleme näitena kaupluse infosüsteemi. Kaupluse infosüsteemi SQL-andmebaasis on loodud tabelid *Kaup* ja *Kauba_seisundi_liik* (seisundi klassifikaatori tabel).

```
CREATE OR REPLACE TRIGGER kaup_i BEFORE INSERT ON Kaup
FOR EACH ROW
WHEN (NOT (NEW.kauba_seisundi_liik_kood=1))
BEGIN
/*Triger, mis kontrollib, et kaup luuakse õiges seisundis (seisundis
koodiga 1)*/
     RAISE_APPLICATION_ERROR(-20002, 'Kaup luuakse ebakorrektses
seisundis!');
END;
/

CREATE OR REPLACE TRIGGER kaup_u BEFORE UPDATE OF
kauba_seisundi_liik_kood ON Kaup
FOR EACH ROW
WHEN (NOT
((OLD.kauba_seisundi_liik_kood=NEW.kauba_seisundi_liik_kood) OR
     (OLD.kauba_seisundi_liik_kood=1 AND
NEW.kauba_seisundi_liik_kood=2) OR
     (OLD.kauba_seisundi_liik_kood=2 AND
NEW.kauba_seisundi_liik_kood=1) OR
     (OLD.kauba_seisundi_liik_kood IN (1, 2) AND
NEW.kauba_seisundi_liik_kood=3)))
BEGIN
/*Triger, mis kontrollib seisundimuudatuste korrektsust
OLD.kauba_seisundi_liik_kood=NEW.kauba_seisundi_liik_kood
tähendab, et muudatus seisundi koodi veerus, mille tulemusena
seisundi kood ei muutu, on lubatud.*/
     RAISE_APPLICATION_ERROR(-20003, 'Kauba ebakorrektne
seisundimuudatus!');
END;
/

CREATE OR REPLACE TRIGGER kaup_d BEFORE DELETE ON Kaup
BEGIN
/*Triger, mis ei lase kaupade andmeid füüsiliselt andmebaasist
kustutada. Ka see triger on tuletatud seisundidiagrammist, kuna
seisundidiagrammi kohaselt ei kustutata kaupade andmeid füüsiliselt
andmebaasist. Kauba kustutamisel muudetakse selle seisundit, kuid
kauba andmed jäävad andmebaasi.

Triger luuakse lausetaseme trigerina, kuna mistahes kauba andmeid ei
tohi füüsiliselt kustutada ja seega ei ole põhjust trigeri
protseduuri iga kustutatava rea korral eraldi käivitada.*/
     RAISE_APPLICATION_ERROR(-20004, 'Kaupade andmeid ei tohi
andmebaasist kustutada!');
END;
/
```

```
/*Juhime tähelepanu, et erineva sisuga veateadetes tuleks kasutada
erinevaid vea koode.*/

INSERT INTO Kaup(kauba_kood, pakendi_tyyp_kood,
kauba_seisundi_liik_kood, nimetus, hetke_hind)
VALUES ('A12312A',1,2,'Suur asi',100);
--Lisamine ebaõnnestus
--SQL Error: ORA-20002: Kaup luuakse ebakorrektses seisundis!

INSERT INTO Kaup(kauba_kood, pakendi_tyyp_kood,
kauba_seisundi_liik_kood, nimetus, hetke_hind)
VALUES ('A12312A',1,1,'Suur asi',100);
--Lisamine õnnestus

UPDATE Kaup SET kauba_seisundi_liik_kood=2
WHERE kauba_kood='A12312A';
--Muudatus õnnestus

UPDATE Kaup SET kauba_seisundi_liik_kood=1
WHERE kauba_kood='A12312A';
--Muudatus õnnestus

UPDATE Kaup SET kauba_seisundi_liik_kood=3
WHERE kauba_kood='A12312A';
--Muudatus õnnestus

UPDATE Kaup SET kauba_seisundi_liik_kood=1
WHERE kauba_kood='A12312A';
--Muudatus ebaõnnestus
--SQL Error: ORA-20003: Kauba ebakorrektne seisundimuudatus!

DELETE FROM Kaup;
--Kustutamine ebaõnnestus
/*SQL Error: ORA-20004: Kaupade andmeid ei tohi andmebaasist
kustutada!*/
```

Nimi: Põhiolemite seisundimuudatuste ajaloo registreerimine

Ingliskeelne nimi: Registering the history of state changes of the main entities

Eelmised mustrid: Põhiolemite hetkeseisundi registreerimise vajadus

Probleem: Kuidas modelleerida infosüsteemi detailanalüüsi dokumendis vajadust säilitada andmebaasis andmeid põhiolemitüüpi kuuluvate olemite (põhiolemite) seisundimuudatuste ajaloo kohta?

Kontekst:

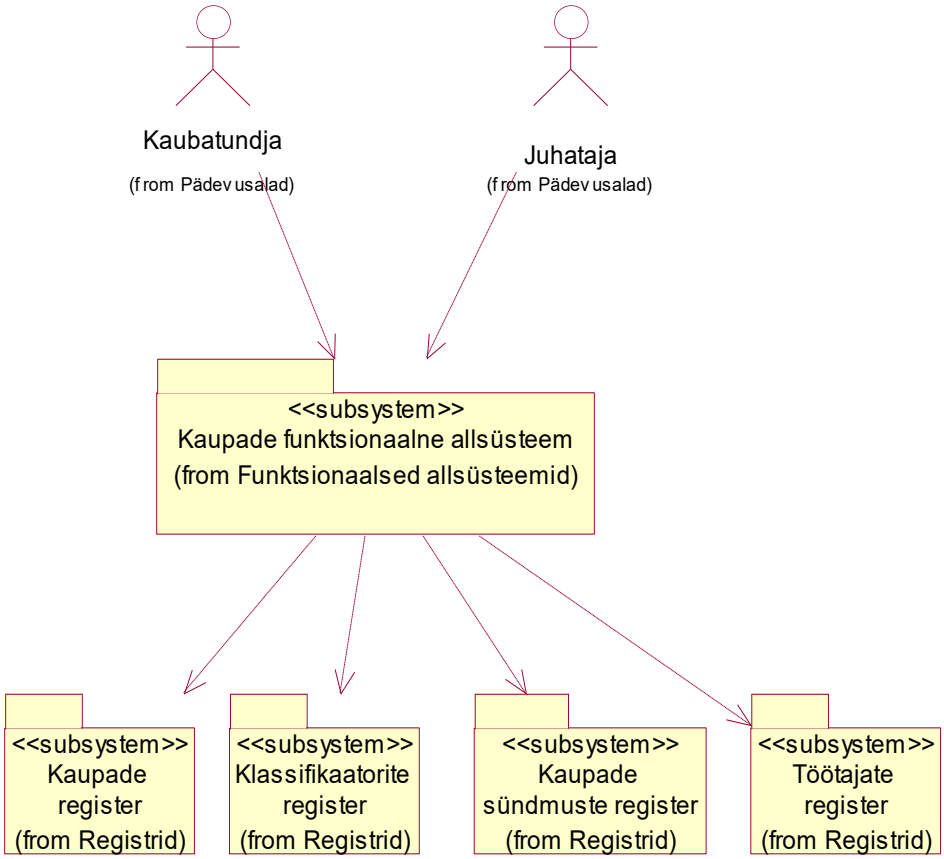
- Detailanalüüsi mudelis on fikseeritud põhiolemite hetkeseisundi registreerimise vajadus.
- Detailanalüüsi kontseptuaalses andemudelis kirjeldatakse põhiolemitüübile vastav seisundi klassifikaator.
- Järelevalve teostamiseks ning võimalike rikkumiste avastamiseks või mõnel muul põhjusel on vajalik teada, kes vastutab konkreetse seisundimuudatuse eest ning millisel ajal seisundimuudatus toimus.

Lahendus: Modelleerida iga põhiolemitüübi korral, mille puhul on vajalik säilitada seisundimuudatuste ajalugu, eraldi seisundimuudatuste ajaloo olemitüüp. Seisundimuudatust võib käsitleda kui sündmust (võib ka kasutada terminit *kanne*), mis toimus mingil ajahetkel, mingi osapoole osalusel ning mille tulemusena muutus põhiolemi seisund. Igale sellisele seisundimuudatuse ajaloo olemitüübile vastab infosüsteemi äriarhitektuuris eraldi register. Seda registrit haldab sama funktsionaalne allsüsteem, mis haldab ka põhiolemitüübile vastavat registrit.

Kuna sellise lahenduse kasutuselevõtt suurendab süsteemi keerukust ning kogutavate andmete hulka, siis tuleks seda kasutada vaid juhul, kui kogutavaid seisundimuudatuste ajaloo andmeid on plaanis hakata reaalselt kasutama. Sündmuste registris olevaid andmeid saab kasutada protsesside analüüsimiseks ning visualiseerimiseks, et leida näiteks „pudelikaelad“ (tööprotsess takerdub mingil sammul kellegi või millegi taha) ja üritada seejärel protsessid sujuvamaks muuta.

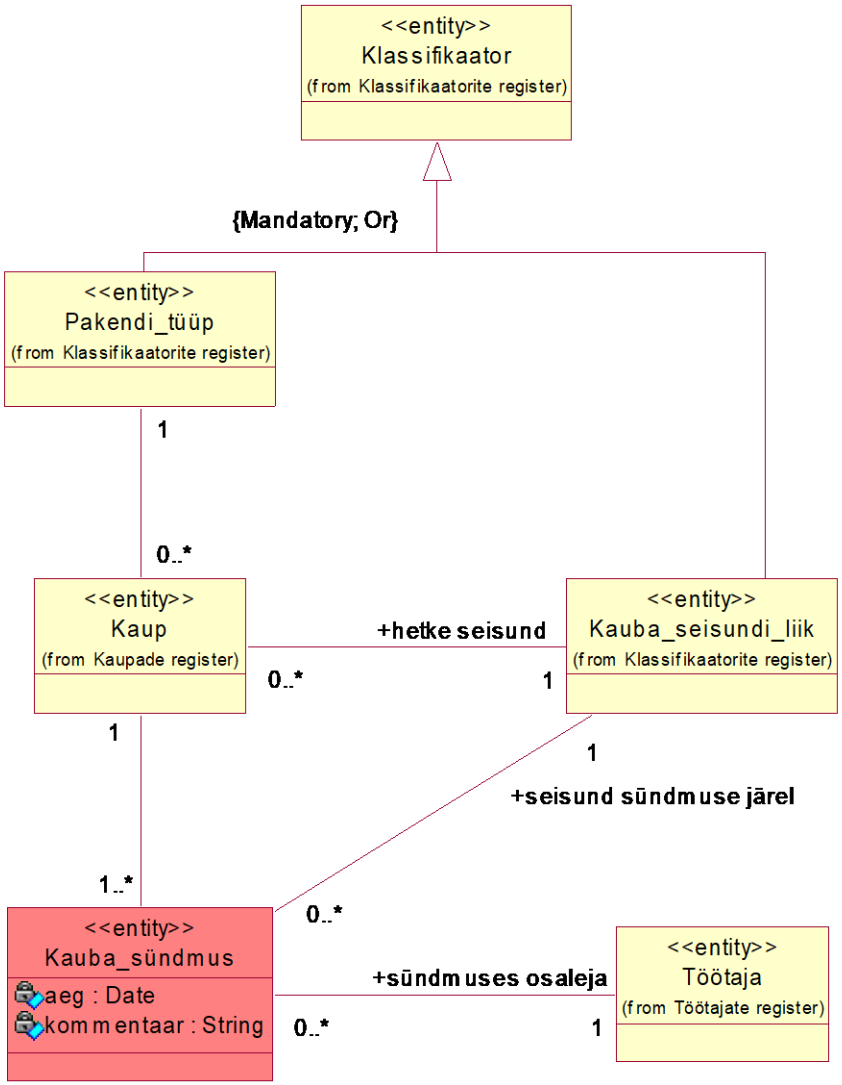
Näide: Vaatleme näitena kaupluse infosüsteemi, mille korral on üheks põhiolemitüübiks *Kaup.* Põhiolemitüübile *Kaup* vastab infosüsteemi äriarhitektuuris *kaupade register*. Kuna peetakse vajalikuks säilitada kaupade seisundimuudatuste ajalugu, siis on äriarhitektuuris ette nähtud ka *kaupade sündmuste register*. Äriarhitektuuri kohaselt teenindab *kaupade registrit* ja *kaupade sündmuste registrit* (lisab/loeb/muudab/kustutab nendes andmeid)

kaupade funktsionaalne allsüsteem. *Kaupade funktsionaalne allsüsteem* peab toimimiseks lugema andmeid ka teistest registritest – antud näite korral *klassifikaatorite registrist* ja *töötajate registrist*. *Kaupade funktsionaalne allsüsteem* pakub teenuseid *kaubatundja* ja *juhataja* pädevusalade esindajatele. Joonis 29 esitatakse infosüsteemi äriarhitektuuri fragment.



Joonis 29 Kaupluse infosüsteemi äriarhitektuuri fragment.

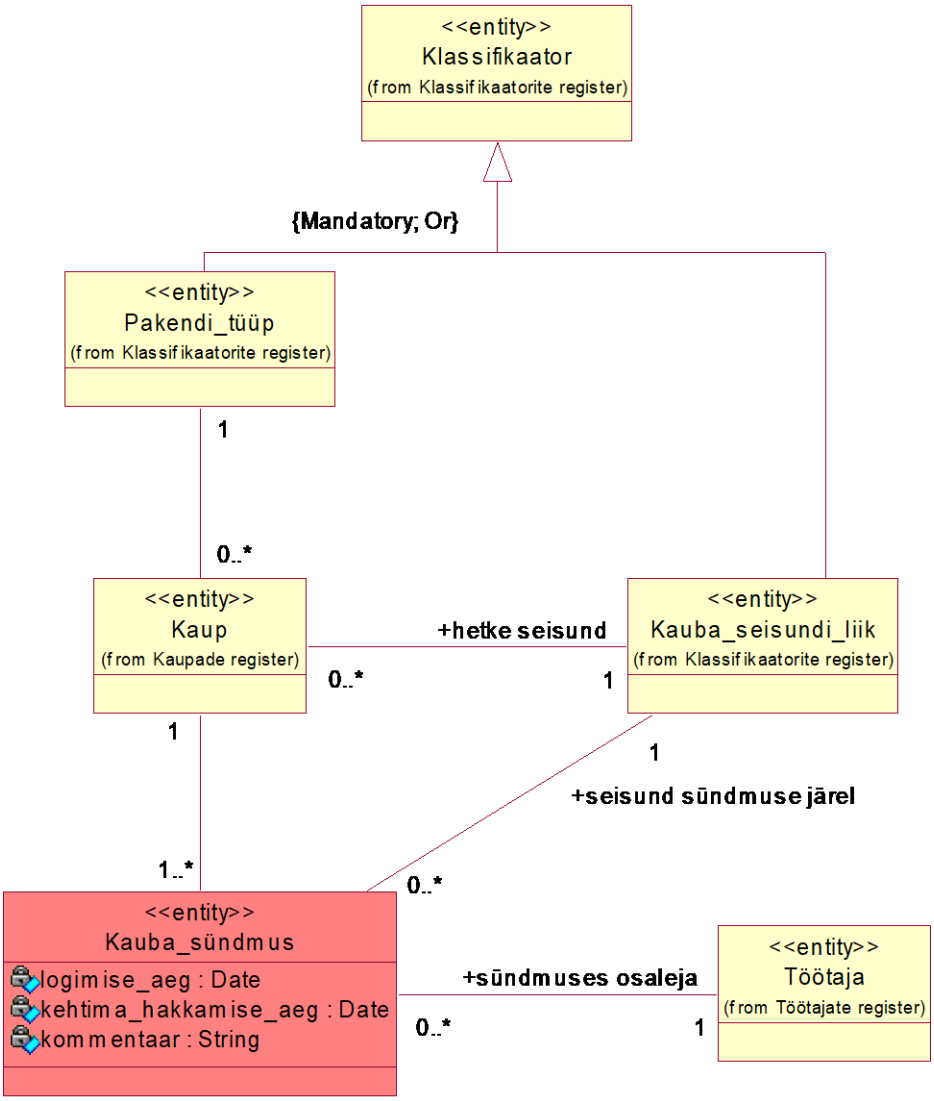
Joonis 30 esitab *kaupade sündmuste registri* olemi-suhte diagrammi, milles esitatakse põhiolemitüüp *Kauba_sündmus* ja selle seosed teiste olemitüüpidega. Igal kaubal on vähemalt üks seotud sündmus, sest kauba loomisel läheb kaup seisundisse „Aktiivne“ ja see registreeritakse kauba sündmusena.



Joonis 30 Kaupade sündmuste registri olemi-suhte diagramm.

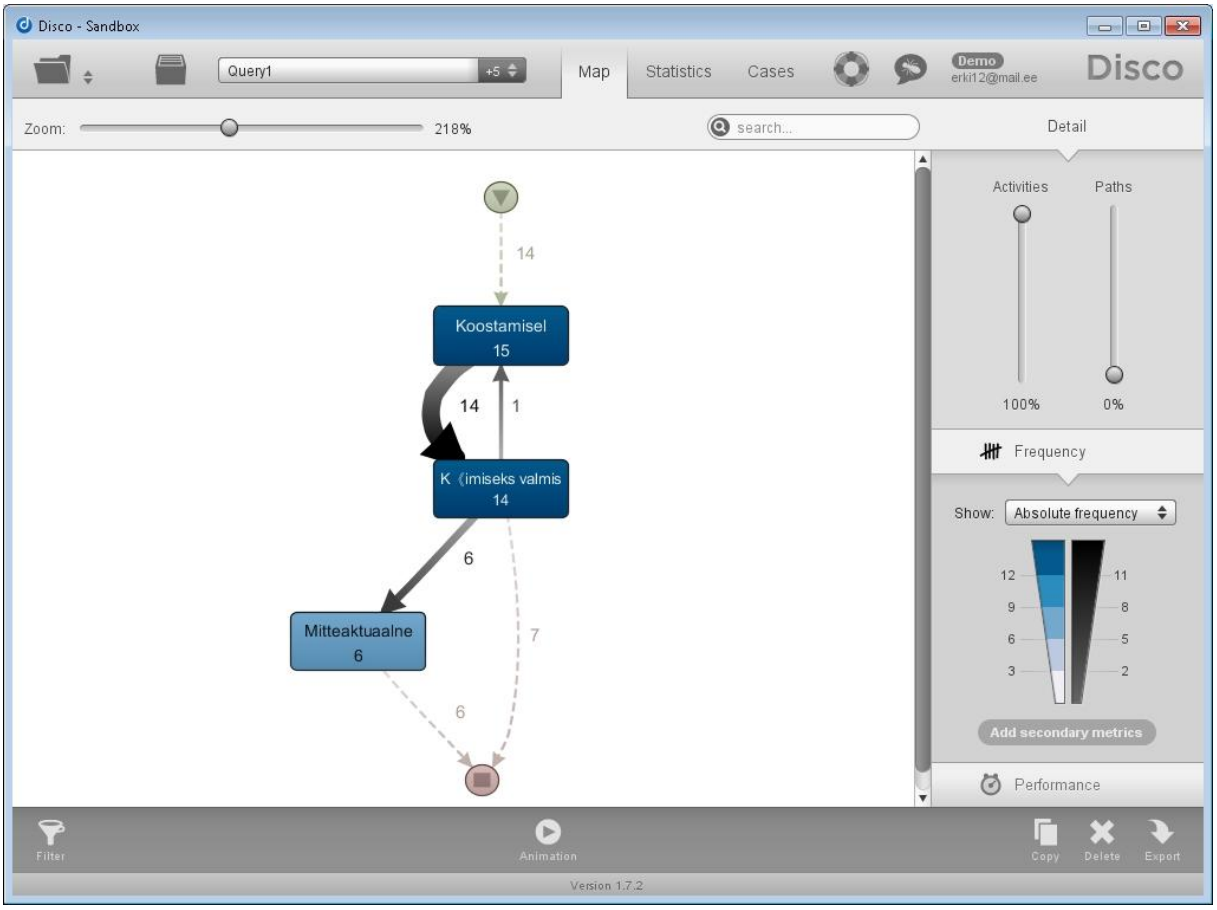
Seisundimuudatuste sündmuste puhul on võimalik eristada sündmuse logimise aega (ingl *transaction time*) ja uue seisundi kehtima hakkamise aega (ingl *valid time*). Joonis 30 on need kaks aega kokku võetud, eeldusel, et uus seisund hakkab kehtima samal hetkel kui süsteemis registreeriti sündmus. Seetõttu on olemitüübil *Kauba_sündmus* üks temporaalne (aja-andmete) atribuut – *aeg*.

Kui logimise aeg ja uue seisundi kehtima hakkamise aeg võivad erineda, siis tuleb kasutada mudelit Joonis 31. Olemitüübil *Kauba_sündmus* on temporaalsed atribuudid *logimise_aeg* ja *kehtima_hakkamise_aeg*.

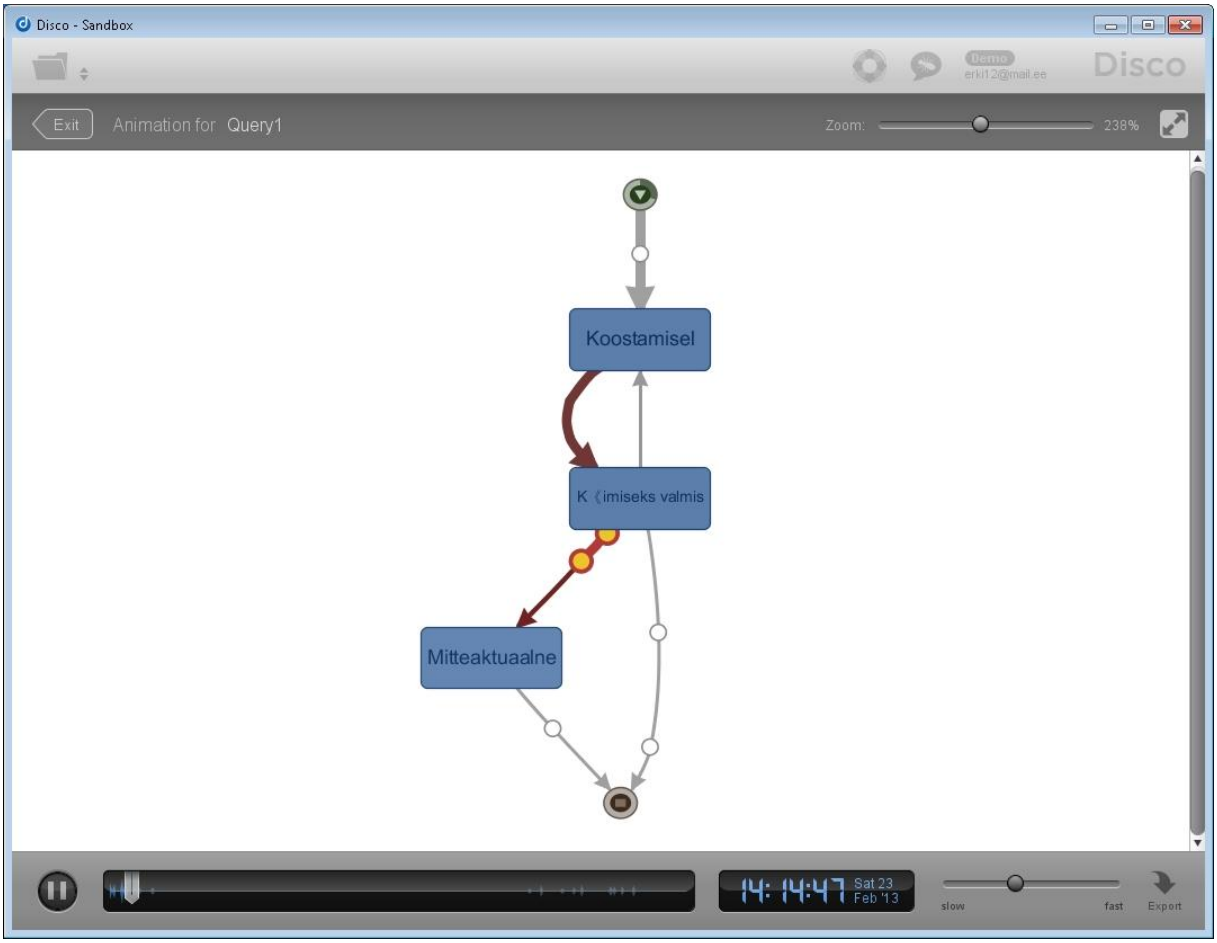


Joonis 31 Kaupade sündmuste registri olemi-suhte diagramm.

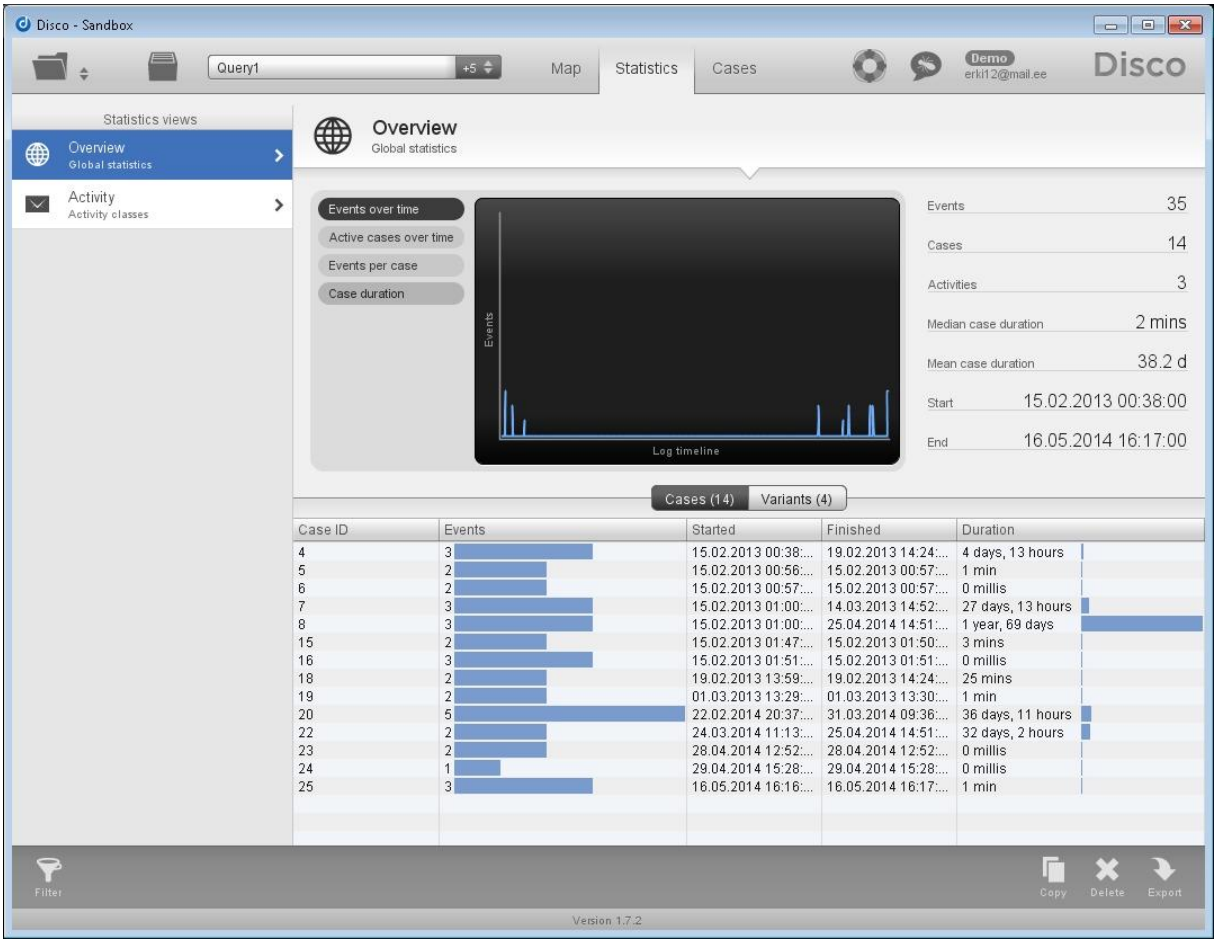
Sündmuste registrites olevate andmete analüüsimiseks ja visualiseerimiseks võib kasutada näiteks programmi *Disco* (https://fluxicon.com/disco/). Joonis 32 – Joonis 34 visualiseerivad andmeid, mis on kogutud ühte konkreetsesse sündmuste registrisse (küsimuste sündmuste register).



Joonis 32 Põhiolemitüübi elutsükli visualiseerimine (programmis Disco).



Joonis 33 Põhiolemite elutsüklite animatsioon (programmis Disco).



Joonis 34 Põhiolemitega toimunud sündmuste statistika (programmis Disco).

Edasiseks lugemiseks

Peek, S., 2019. *Infosüsteemi põhiolemite seisundite esitamine SQL-andmebaasides*. Magistritöö. Tallinna Tehnikaülikooli Tarkvarateaduse instituut. [WWW] https://digikogu.taltech.ee/et/Item/101f40e2-d9fe-4bef-ab25-54508f18c078

Töös kirjeldatakse mustri formaadis kuut lahendust kuidas hoida seisundite andmeid SQL-andmebaasis ning viidatakse nende variatsioonidele. Töös selgitatakse, kuidas iga pakutud disaini kohandada tüüpiliste nõuetega, mis võivad esineda seisundite kohta arvet pidavates süsteemides. Töö teises osas viiakse nende disainide võrdlemiseks läbi eksperiment. Eksperimendi käigus võrreldakse kõiki töös esitatud põhidisaine nii päringute ja andmemuudatus operatsioonide täitmiskiiruse ja keerukuse, seisundite hulga muutmise keerukuse kui ka andmemahtude alusel.

Volostnõh, P., 2023. *Design and implementation of a tool for generating PostgreSQL functions based on contracts of database operations*. Magistritöö. Tallinna Tehnikaülikooli Tarkvarateaduse instituut. [WWW] https://digikogu.taltech.ee/et/Item/05f2dd7c-e14a-496f-b69f-f74a5d05dc09

Selle töö tegemise käigus kavandati ja realiseeriti tarkvara andmebaasioperatsioonide lepingute põhjal PostgreSQL funktsioonide loomise lausete genereerimiseks. Selle tarkvara loomiseks pidi autor kõigepealt kavandama kaks valdkonnapõhist keelt. Ühe keele abil pannakse kindlas vormis kirja andmebaasioperatsioonide lepingud, sest selliseid vabas vormis kirjeldusi nagu kasutatakse selles mustrite dokumendis ei saa koodi genereerimise lähtepunktina kasutada. Teise keele abil pannakse kirja andmebaasi tabelite struktuur, sest seda infot peab generaator koodi genereerimiseks teadma. Töö tulemusena valmis töötav avatud lähtekoodiga (GPL ver 3 litsents) tarkvara, mis on kättesaadav siit: https://github.com/ghpv/dbcsql Lõputöö dokumendist leiab näiteid nii generaatorile antava sisendi kui ka genereeritud koodi kohta.